
User Manual

for MPC560XB ICU Driver

Document Number: UM13ICUASR4.0R1.0.1
Rev 1.2





Contents

Section number	Title	Page
Chapter 1		
Revision History		
Chapter 2		
Introduction		
2.1	Supported Derivatives.....	11
2.2	Overview.....	12
2.3	About this Manual.....	12
2.4	Acronyms and Definitions.....	13
2.5	Reference List.....	13
Chapter 3		
Driver		
3.1	Requirements.....	15
3.2	Driver Design Summary.....	15
3.3	Hardware channel availability.....	16
3.4	Additional Requirements and Deviations.....	16
3.5	Function Definitions.....	20
3.5.1	Function Icu_Init.....	20
3.5.2	Function Icu_CheckWakeup.....	20
3.5.3	Function Icu_DeInit.....	21
3.5.4	Function Icu_SetMode.....	21
3.5.5	Function Icu_SetClockMode.....	22
3.5.6	Function Icu_GetInputLevel.....	22
3.5.7	Function Icu_DisableWakeup.....	22
3.5.8	Function Icu_EnableWakeup.....	23
3.5.9	Function Icu_SetActivationCondition.....	23
3.5.10	Function Icu_DisableNotification.....	24
3.5.11	Function Icu_EnableNotification.....	24
3.5.12	Function Icu_GetInputState.....	25

Section number	Title	Page
3.5.13	Function Icu_StartTimestamp.....	25
3.5.14	Function Icu_StopTimestamp.....	26
3.5.15	Function Icu_GetTimestampIndex.....	26
3.5.16	Function Icu_ResetEdgeCount.....	27
3.5.17	Function Icu_EnableEdgeCount.....	27
3.5.18	Function Icu_EnableEdgeDetection.....	28
3.5.19	Function Icu_DisableEdgeCount.....	28
3.5.20	Function Icu_DisableEdgeDetection.....	29
3.5.21	Function Icu_GetEdgeNumbers.....	29
3.5.22	Function Icu_StartSignalMeasurement.....	29
3.5.23	Function Icu_StopSignalMeasurement.....	30
3.5.24	Function Icu_GetTimeElapsed.....	30
3.5.25	Function Icu_GetDutyCycleValues.....	31
3.5.26	Function Icu_GetVersionInfo.....	31
3.5.27	Function Icu_GetPulseWidth.....	32
3.5.28	Function Icu_LLD_StartSignalMeasurement.....	32
3.6	Structure Definitions.....	33
3.6.1	Structure Icu_ConfigType.....	33
3.6.2	Structure Icu_ChannelConfigType.....	34
3.6.3	Structure Icu_DutyCycleType.....	35
3.6.4	Structure Std_VersionInfoType.....	35
3.7	Enum Definitions.....	36
3.7.1	Enumeration Icu_SelectPrescalerType.....	36
3.7.2	Enumeration Icu_LevelType.....	36
3.7.3	Enumeration Icu_ActivationType.....	37
3.7.4	Enumeration Icu_InputStateType.....	37
3.7.5	Enumeration Icu_MeasurementModeType.....	37
3.7.6	Enumeration Icu_ModeType.....	37
3.7.7	Enumeration Icu_SignalMeasurementPropertyType.....	38

Section number	Title	Page
3.7.8	Enumeration Icu_TimestampBufferType.....	38
3.7.9	Enumeration Icu_WakeupCapableType.....	38
3.8	Typedef Definitions.....	39
3.8.1	Typedef Icu_EdgeNumberType.....	39
3.8.2	Typedef Icu_ChannelType.....	39
3.8.3	Typedef Icu_ChannelStateType.....	39
3.8.4	Typedef Icu_IndexType.....	39
3.8.5	Typedef Icu_MeasurementSubModeType.....	40
3.8.6	Typedef Icu_NotifyType.....	40
3.8.7	Typedef Icu_ValueType.....	40
3.8.8	Typedef Icu_ParamType.....	40
3.9	Define Definitions.....	40
3.9.1	Define ICU_CHECKWAKEUP_ID.....	40
3.9.2	Define ICU_DEINIT_ID.....	40
3.9.3	Define ICU_DISABLEEDGECOUNT_ID.....	41
3.9.4	Define ICU_DISABLEEDGEDETECTION_ID.....	41
3.9.5	Define ICU_DISABLENOTIFICATION_ID.....	41
3.9.6	Define ICU_DISABLEWAKEUP_ID.....	41
3.9.7	Define ICU_ENABLEEDGECOUNT_ID.....	41
3.9.8	Define ICU_ENABLEEDGEDETECTION_ID.....	41
3.9.9	Define ICU_ENABLENOTIFICATION_ID.....	42
3.9.10	Define ICU_ENABLEWAKEUP_ID.....	42
3.9.11	Define ICU_GETDUTYCYCLEVALUES_ID.....	42
3.9.12	Define ICU_GETEDGENUMBERS_ID.....	42
3.9.13	Define ICU_GETINPUTSTATE_ID.....	42
3.9.14	Define ICU_GETTIMEELAPSED_ID.....	42
3.9.15	Define ICU_GETTIMESTAMPINDEX_ID.....	42
3.9.16	Define ICU_GETVERSIONINFO_ID.....	43
3.9.17	Define ICU_INIT_ID.....	43

Section number	Title	Page
3.9.18	Define ICU_RESETEDEGECOUNT_ID.....	43
3.9.19	Define ICU_SET_CLOCK_MODE_ID.....	43
3.9.20	Define ICU_GET_INPUT_LEVEL_ID.....	43
3.9.21	Define ICU_SETACTIVATIONCONDITION_ID.....	43
3.9.22	Define ICU_SETMODE_ID.....	44
3.9.23	Define ICU_STARTSIGNALMEASUREMENT_ID.....	44
3.9.24	Define ICU_STARTTIMESTAMP_ID.....	44
3.9.25	Define ICU_STOPSIGNALMEASUREMENT_ID.....	44
3.9.26	Define ICU_STOPTIMESTAMP_ID.....	44
3.9.27	Define ICU_E_ALREADY_INITIALIZED.....	44
3.9.28	Define ICU_E_BUSY_OPERATION.....	45
3.9.29	Define ICU_E_EDGECOUNTING_OVERFLOW.....	45
3.9.30	Define ICU_E_MEASUREMENT_OVERFLOW.....	45
3.9.31	Define ICU_E_NOT_STARTED.....	45
3.9.32	Define ICU_E_PARAM_ACTIVATION.....	45
3.9.33	Define ICU_E_PARAM_BUFFER_PTR.....	45
3.9.34	Define ICU_E_PARAM_BUFFER_SIZE.....	45
3.9.35	Define ICU_E_PARAM_CHANNEL.....	46
3.9.36	Define ICU_E_PARAM_CLOCK_MODE.....	46
3.9.37	Define ICU_E_PARAM_NOTIFY_INTERVAL.....	46
3.9.38	Define ICU_E_PARAM_VINFO.....	46
3.9.39	Define ICU_E_PARAM_CONFIG.....	46
3.9.40	Define ICU_E_PARAM_MODE.....	47
3.9.41	Define ICU_E_TIMESTAMP_OVERFLOW.....	47
3.9.42	Define ICU_CAPTUREREGISTER_ID.....	47
3.9.43	Define ICU_E_UNINIT.....	47
3.10	Configuration Parameters.....	47
3.10.1	Plugin interface.....	48

Section number	Title	Page
3.10.2	Pre-Compile Parameters.....	50
3.10.2.1	icuallowwkponallchls.....	51
3.10.2.2	IcuChannelId.....	52
3.10.2.3	IcuDeInitApi.....	52
3.10.2.4	IcuDevErrorDetect.....	52
3.10.2.5	IcuDisableWakeupApi.....	53
3.10.2.6	IcuDmaMaxChannel.....	53
3.10.2.7	IcuEdgeCountApi.....	53
3.10.2.8	IcuEdgeDetectApi.....	54
3.10.2.9	IcuEnableDualClockMode.....	54
3.10.2.10	IcuEnableWakeupApi.....	54
3.10.2.11	IcuEXT_ISR_IFCPRDigitalFilter.....	55
3.10.2.12	IcuEXT_ISRaltIFCPRDigitalFilter.....	55
3.10.2.13	IcuGetDutyCycleValuesApi.....	56
3.10.2.14	IcuGetInputLevelApi.....	56
3.10.2.15	IcuGetInputStateApi.....	56
3.10.2.16	IcuGetTimeElapsedApi.....	56
3.10.2.17	IcuGetVersionInfoApi.....	57
3.10.2.18	IcuHwChannel.....	57
3.10.2.19	IcuIndex.....	59
3.10.2.20	IcuMaxChannel.....	59
3.10.2.21	IcuOverflowNotificationApi.....	59
3.10.2.22	IcuReportWakeupSource.....	60
3.10.2.23	IcuSetModeApi.....	60
3.10.2.24	IcuSignalMeasurementApi.....	60
3.10.2.25	IcuTimestampApi.....	61
3.10.2.26	IcuWakeupFunctionalityApi.....	61
3.10.3	Link Time parameters.....	61

Section number	Title	Page
3.10.4	Post - Build Parameters.....	61
3.10.4.1	ICU External Interrupt Filter Setting.....	62
3.10.4.2	ICU External Enable Interrupt Filter.....	63
3.10.4.3	IcuChannelWakeupInfo.....	63
3.10.4.4	IcuDefaultStartEdge.....	64
3.10.4.5	icudisableecumwakeupsourcenotification.....	65
3.10.4.6	IcuDMAEnable.....	66
3.10.4.7	IcuEmiosBusSelect.....	66
3.10.4.8	IcuEmiosDigitalFilter.....	67
3.10.4.9	IcuEmiosFreeze.....	68
3.10.4.10	IcuEmiosPrescaler_Alternate.....	68
3.10.4.11	IcuEmiosPrescaler.....	69
3.10.4.12	IcuMeasurementMode.....	70
3.10.4.13	Icu Overflow Notification.....	71
3.10.4.14	Icu_ParamType.....	71
3.10.4.15	IcuSignalMeasurementProperty.....	73
3.10.4.16	IcuSignalMeasureWithoutInterrupt.....	73
3.10.4.17	IcuSignalNotification.....	74
3.10.4.18	IcuTimestampMeasurementProperty.....	75
3.10.4.19	IcuTimestampNotification.....	75
3.10.4.20	IcuUserModeForDutycycle.....	76
3.10.4.21	IcuWakeupCapability.....	76
3.10.4.22	IcuWKPU_ISR_WIPUER.....	77

Chapter 1

Revision History

Table 1-1. Revision History

Revision	Date	Author	Description
1.0	20/01/2012	Syed Hussaini	Updated for MPC560xB BETA 0.9.0 Release
1.1	13/05/2013	Syed Hussaini	1.0.0 Release
1.2	06/June/2014	Hai Pham Minh	1.0.1 Release



Chapter 2

Introduction

This User Manual describes Freescale Semiconductor AUTOSAR microcontroller abstraction layer (MCAL) Input Capture Unit (ICU) for MPC560XB microcontroller.

AUTOSAR ICU driver configuration parameters and deviations from the specification are described in the Driver chapter of this document. AUTOSAR ICU driver requirements and APIs are described in the AUTOSAR ICU driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of Freescale Semiconductor:

Table 2-1. MPC560XB Supported Derivatives

Freescale Semiconductor	mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100 mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100 mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144 mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64 mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208 mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100 mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bkll_lqfp100
-------------------------	--

Table 2-1. MPC560XB Supported Derivatives

	mpc5606bxlq_lqfp144 mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176 mpc5607bxmg_mapbga208
--	--

All of the above microcontroller devices are collectively named as **MPC560XB**.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About this Manual

This Technical Reference employs the following typographical conventions:

Boldface type: Bold is used for important terms, notes and warnings.

Italic font: Italic typeface is used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

NOTE

This is a note.

2.4 Acronyms and Definitions

Table 2-2. Acronyms and Definitions

Abbreviation and Definitions	Description
BSW	Basic Software
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECU	Electronic Control Unit
ICU	Input Capture Unit
ISR	interrupt Service Routine
OS	Operating System
RAM	Random Access Memory
ROM	Read-only Memory
MCU	Microcontroller Unit
GUI	Graphical User Interface
EcuM	ECU state Manager
API	Application Programming Interface
PB Variant	Post Build Variant
PC Variant	Pre Compile Variant

2.5 Reference List

Table 2-3. Reference List

#	Title	Version
1	AUTOSAR 4.0 Rev0003ICU Driver Software Specification Document.	4.2.0 Rev 0003
2	MPC5604B Reference Manual	Rev. 8.2, September 2013
3	MPC5607B Reference Manual	Rev. 7.2, May 2012
4	5606B Reference Manual	Rev. 2, May 2014
5	5602D Reference Manual	Rev. 4.2, Dec 2013
6	Bolero 1.5M cut 2.2 Errata Sheet (MPC5607B_1M03Y)	Rev. 12Dec2013

Table continues on the next page...

Table 2-3. Reference List (continued)

#	Title	Version
7	Bolero 512k cut 2.3 Errata Sheet (MPC5604B_2M27V)	Rev. 26 FEB 2014
8	Bolero 1M cut 1.0 Errata Sheet (MPC5606B_0N13E)	Rev. , 12 DEC 2013
9	Bolero 256k cut 1.1 Errata Sheet (MPC5602D_1M18Y)	Rev. , 12 DEC 2013

Chapter 3 Driver

3.1 Requirements

Requirements for this driver are detailed in the AUTOSAR 4.0 Rev0003ICU Driver Software Specification document (See Table [Reference List](#)).

3.2 Driver Design Summary

The ICU Driver controls the input capture of the microcontroller. It provides the following features:

- High time / Low time measurement
- Duty Cycle measurement
- Period time measurement
- Edge detection and notification
- Edge counting (with or without hardware gating)
- Edge time stamping
- Wake-up interrupts

For signal edge detection, the edge detector of a capture compare unit or the interrupt controller for external events are used.

For signal measuring a capture timer and at least one capture register are needed.

The EMIOS module of MPC560XB supports period time measurement, edge detection and notification, edge counting and edge time stamping.

The SIUL module of MPC560XB supports edge detection and notification.

The WKPU module of MPC560XB supports edge detection and notification. For details please refer to [Table 3-1](#).

The ICU driver provides an optional API and configuration parameters for changing the base clock of the controlled hardware. A dual clock functionality is offered by switching between two configured values of the clock prescaler.

For each user configured channel, a symbolic name is generated by the Tresos Studio configuration tool. The name shall be consequently used in upper applications (e.g. ICU_EMIO0_0_CH_1).

By default all channels offer interrupt handlers. For each channel not configured by the user in Tresos Studio configuration tool, the code for interrupt handling is removed based on a series of ifdefs.

3.3 Hardware channel availability

Table 3-1. ICU Hardware channels availability for MPC560XB family

Device	Total EMIO channels	Total SIUL channels	Total WKPU channels
mpc5644b_lqfp176	63 ch	24 ch	29 ch
mpc5644b_lqfp208	64 ch	24 ch	30 ch
mpc5644c_bga256	64 ch	24 ch	30 ch
mpc5644c_lqfp176	63 ch	24 ch	29 ch
mpc5644c_lqfp208	64 ch	24 ch	30 ch
mpc5645b_lqfp176	63 ch	24 ch	29 ch
mpc5645b_lqfp208	64 ch	24 ch	30 ch
mpc5645c_bga256	64 ch	24 ch	30 ch
mpc5645c_lqfp176	63 ch	24 ch	29 ch
mpc5646b_lqfp176	63 ch	24 ch	29 ch
mpc5646b_lqfp208	64 ch	24 ch	30 ch
mpc5646c_bga256	64 ch	24 ch	30 ch
mpc5646c_lqfp176	63 ch	24 ch	29 ch
mpc5646c_lqfp208	64 ch	24 ch	30 ch

3.4 Additional Requirements and Deviations

Deviations from the AUTOSAR ICU Driver software specification are listed in table **Deviations Status Column Description**.

Table 3-2. Deviations Status Column Description

Term	Definition
N/A	Not available
N/T	Not testable
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently, or out of scope for the ICU driver.

Table 3-3. ICU Deviations Table

Requirement	Status	Description	Notes
ICU003	N/I	Production errors shall be reported to the Diagnostic Event Manager (DEM).	No Production Error
ICU039	N/F	Configuration of Signal Measurement - This container contains the configuration (parameters) in case the measurement mode is "ICU_MODE_SIGNAL_MEASUREMENT" The definition for each Channel shall contain: Common parameters § Wakeup capability (true / false) § Default Start Edge § Hardware Specific Settings per channel § Measurement Mode - Signal Edge Detection / Notification - Signal Measurement - Timestamp - Edge Counter Specific parameters § If measurement mode is "signal edge detection" the notification function for signal notification shall be configurable § If measurement mode is "signal measurement" the property that could be measured shall be configurable. The values shall be: - High Time - Low Time - Period Time - Duty Cycle Values (High Time and Period Time) § If measurement mode is "timestamp measurement", buffer handling shall be configurable. The values shall be: - Circular buffer handling - Linear buffer handling Also the notification function for notifying the number of requested timestamps shall be configurable § If measurement mode is "edge counter", the counting mode (activation edge) shall be configurable. The values shall be: - Rising Edge - Falling Edge - Both edges § If the channel is configured as wakeup capable, - the callout function for validation of wakeup reason shall be configurable - The value transmitted to the EcuM shall be configurable § Assigned capture register(s) (can also be none for channels which provide only edge detection like an external interrupt) Assigned capture timer (can also be none for channels	For memory space saving capture register and capture timer were omitted, indexed access is used. Capture registers and capture timer are not enough for this architecture, it would require also the control register and the status register. Because registers in memory are very well organized, the access to these registers was done by macros that access memory with offset depending the channel used.

Table continues on the next page...

Table 3-3. ICU Deviations Table (continued)

Requirement	Status	Description	Notes
		which provide only edge detection like an external interrupt)	
ICU052	N/S	Icu_Init - If the register can affect several hardware modules and if it is an I/O register it shall be initialized by the PORT driver.	Not an ICU driver requirement
ICU053	N/S	Icu_Init - If the register can affect several hardware modules and if it is not an I/O register it shall be initialized by the MCU driver.	Not an ICU driver requirement
ICU113	N/I	The detection of production code errors cannot be switched off. Specification of ICU Driver	No Production Error
ICU116	N/I	The module shall include the Dem.h file. By this inclusion the API's to report errors as well as the required Event Id symbols are included. This specification defines the name of the Event Id symbols which are provided by XML to the DEM configuration tool. The DEM configuration tool assigns ECU dependent values to the Event Id symbols and publishes the symbols in Dem_IntErrId.h.	No Production Error
ICU117	N/I	Values for production code event ID's are assigned externally by the configuration of the DEM. They are published in the file Dem_IntErrId.h and included via Dem.h.	No Production Error
ICU128	N/S	One-time writable registers that require initialization directly after reset shall be initialized by the startup code.	Not an ICU driver requirement
ICU129	N/S	All other registers shall be initialized by the startup code.	Not an ICU driver requirement
ICU131	N/I	The ICU driver shall describe which other modules (in which versions) are required. This description shall be done by the implementer.	Port Driver Mcu Driver (FMPLL)
ICU227_Conf	N/F	The parameter ICU_ACTIVE_TIME is replaced by ICU_DUTY_CYCLE.	The parameter ICU_DUTY_CYCLE describes better the property that is measured

The following picture shows the hardware limitation for and ICU channel configured as ICU_MODE_SIGNAL_MEASUREMENT, ICU_DUTY_CYCLE, with EMIO unified channel that supports IPWM, that is resulting in a delayed measurement

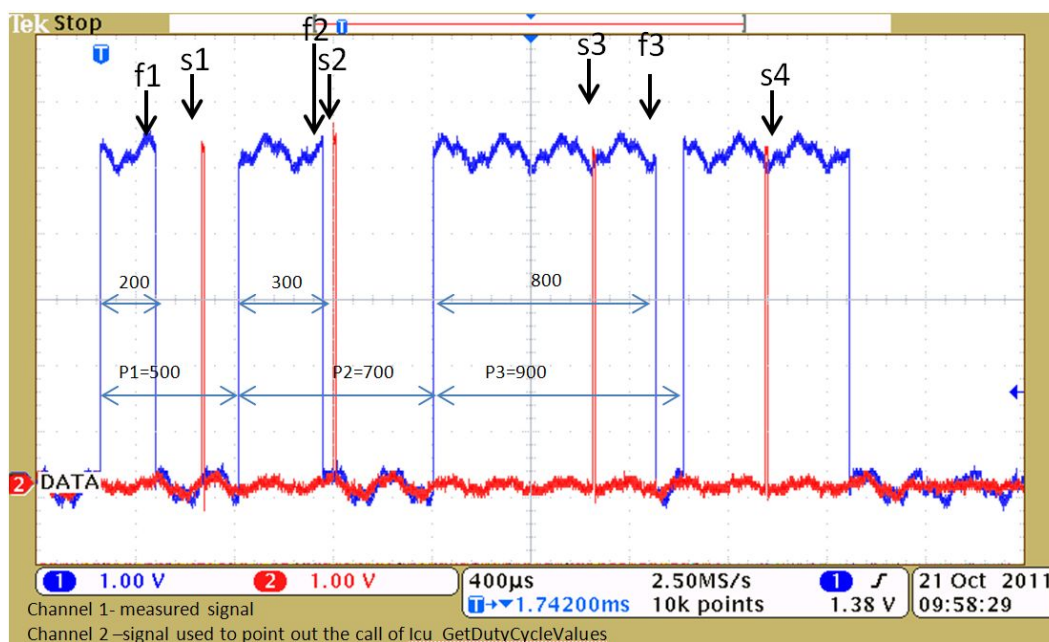


Figure 3-1. Duty cycle measurement in IPWM mode hardware limitation

Explanation:

f1,f2,f3->Falling edges on which the EMIOS IP with unified channel configured as IPWM generates interrupt. In this interrupt values of the counter, captured in register EMIOSB[n] for the rising edge and EMIOSA[n] for the falling edge are read and the ActiveTime is calculated as EMIOSA[n] - EMIOSB[n]. Also here the value of EMIOSB[n] from the previous interrupt is used to compute the PeriodTime of the signal. So when f2 occurs the value captured in EMIOSB[n] is used with the value of EMIOSB[n] saved when f1 occurred to calculate the period of the signal P1=500. Because the information from reg A and B comes in the interrupt generated at the falling edge, when sampling s3(see figure above) Icu_GetDutyCycleValues returns zero instead of ActiveTime=300 and PeriodTime=700 of P2.

The issue here is that although duty cycle information should be available after each period completion (each rising edge if this is the configured activation edge), the actual data is recorded in the driver only after the next falling edge (more generally, the edge opposite to the activation edge) because this is the edge that triggers the interrupt.

When the sampling of the values is done before the falling edge (in this case s3 is before f3), it will return the values recorded from the previous period (this is why s4 reports values from P2 instead of P3; sampling s3 reports values from P1 instead of P2 - but these values have already been returned by the sampling in s2 and the internal buffer of the driver has been cleared, so in the end sampling s3 returns 0). When the sampling of the values is done after the falling edge of the next pulse, it returns the correct values (the ones recorded for the last complete period): sampling s2 returns the correct values for P1(200, 500).

3.5 Function Definitions

APIs of all functions supported by the driver are as per AUTOSAR ICU Driver software specification Version 4.0 Rev0003.

3.5.1 Function Icu_Init

This function initializes the driver.

Prototype: `void Icu_Init(const Icu_ConfigType *ConfigPtr);`

Table 3-4. Icu_Init Arguments

Type	Name	Direction	Description
const Icu_ConfigType *	ConfigPtr	input	Pointer to a selected configuration structure.

Return: void

This service is a non reentrant function used for driver initialization. The Initialization function shall initialize all relevant registers of the configured hardware with the values of the structure referenced by the parameter ConfigPtr. If the hardware allows for only one usage of the register, the driver module implementing that functionality is responsible for initializing the register. The initialization function of this module shall always have a pointer as a parameter, even though for Variant PC no configuration set shall be given. Instead a NULL pointer shall be passed to the initialization function. The Icu module environment shall not call Icu_Init during a running operation (e. g. timestamp measurement or edge counting).

3.5.2 Function Icu_CheckWakeup

Checks if a wakeup capable ICU channel is the source for a wakeup event.

Prototype: `void Icu_CheckWakeup(EcuM_WakeupSourceType WakeupSource);`

Table 3-5. Icu_CheckWakeup Arguments

Type	Name	Direction	Description
EcuM_WakeupSourceType	WakeupSource	input	Information on wakeup source to be checked.

Return: void

The function calls the ECU state manager service EcuM_SetWakeupEvent in case of a valid ICU channel wakeup event.

pre:: Icu_Init must be called before. The channel must be configured as wakeup capable.

3.5.3 Function Icu_DeInit

This function de-initializes the ICU module.

Prototype: void Icu_DeInit(void);

Return: void

This service is a Non reentrant function used for ICU De-Initialization After the call of this service, the state of the peripherals used by configuration shall be the same as after power on reset. Values of registers which are not writable are excluded. This service shall disable all used interrupts and notifications. The Icu module environment shall not call Icu_DeInit during a running operation (e. g. timestamp measurement or edge counting)

pre:: Icu_Init must be called before.

3.5.4 Function Icu_SetMode

This function sets the ICU mode.

Prototype: void Icu_SetMode(Icu_ModeType Mode);

Table 3-6. Icu_SetMode Arguments

Type	Name	Direction	Description
Icu_ModeType	Mode	input	Specifies the operation mode

Return: void

This service is a non reentrant function used for ICU mode selection. This service shall set the operation mode to the given mode parameter. This service can be called during running operations. If so, an ongoing operation that generates interrupts on a wakeup capable channel like e.g. time stamping or edge counting might lead to the ICU module not being able to properly enter sleep mode. This is then a system or ECU configuration issue not a problem of this specification.

pre:: Icu_Init must be called before.

3.5.5 Function Icu_SetClockMode

This function changes the channel prescaler.

Prototype: void Icu_SetClockMode(Icu_SelectPrescalerType Prescaler);

Table 3-7. Icu_SetClockMode Arguments

Type	Name	Direction	Description
Icu_SelectPrescalerType	Prescaler	input	- Prescaler type (Normal or Alternate)
Icu_SelectPrescalerType	Prescaler	input	- Prescaler type (Normal or Alternate)

Return: void

This function sets all channels prescalers based on the input mode.

pre:: Icu_Init must be called before.

3.5.6 Function Icu_GetInputLevel

This function changes the channel prescaler.

Prototype: void Icu_GetInputLevel(uint8 Channel);

Table 3-8. Icu_GetInputLevel Arguments

Type	Name	Direction	Description
uint8	Channel	input	- Numeric identifier of the ICU channel

Return: Icu_LevelType -Level of the input pin.

This function returns the actual status of PIN.

pre:: Icu_Init must be called before.

3.5.7 Function Icu_DisableWakeup

This function disables the wakeup capability of a single ICU channel.

Prototype: void Icu_DisableWakeup(Icu_ChannelType Channel);

Table 3-9. Icu_DisableWakeup Arguments

Type	Name	Direction	Description
Icu_ChannelType	Channel	input	Numeric identifier of the ICU channel

Return: void

This service is reentrant function and shall disable the wakeup capability of a single ICU channel. This service is only feasible for ICU channels configured statically as wakeup capable true. The function Icu_DisableWakeup shall be pre compile time configurable On/Off by the configuration parameter IcuDisableWakeupApi.

pre:: Icu_Init must be called before.

3.5.8 Function Icu_EnableWakeup

This function (re-)enables the wakeup capability of the given ICU channel.

Prototype: void Icu_EnableWakeup(Icu_ChannelType Channel);

Table 3-10. Icu_EnableWakeup Arguments

Type	Name	Direction	Description
Icu_ChannelType	Channel	input	Numeric identifier of the ICU channel

Return: void

The function is reentrant and re-enable the wake-up capability of a single ICU channel.

pre:: Icu_Init must be called before. The channel must be configured as wakeup capable.

3.5.9 Function Icu_SetActivationCondition

This function sets the activation-edge for the given channel.

Prototype: void Icu_SetActivationCondition(Icu_ChannelType Channel, Icu_ActivationType Activation);

Table 3-11. Icu_SetActivationCondition Arguments

Type	Name	Direction	Description
Icu_ChannelType	Channel	input	Numeric identifier of the ICU channel
Icu_ActivationType	Activation	input	Type of activation.

Return: void

This service is reentrant and shall set the activation-edge according to Activation parameter for the given channel. This service shall support channels which are configured for the following Icu_MeasurementMode:

- ICU_MODE_SIGNAL_EDGE_DETECT
- ICU_MODE_TIMESTAMP
- ICU_MODE_EDGE_COUNTER

pre:: Icu_Init must be called before. The channel must be properly configured (ICU_MODE_SIGNAL_EDGE_DETECT, ICU_MODE_TIMESTAMP, ICU_MODE_EDGE_COUNTER).

3.5.10 Function Icu_DisableNotification

This function disables the notification of a channel.

Prototype: void Icu_DisableNotification(Icu_ChannelType Channel);

Table 3-12. Icu_DisableNotification Arguments

Type	Name	Direction	Description
Icu_ChannelType	Channel	input	Numeric identifier of the ICU channel

Return: void

This function is reentrant and disables the notification of a channel.

pre:: Icu_Init must be called before.

3.5.11 Function Icu_EnableNotification

This function enables the notification on the given channel.

Prototype: void Icu_EnableNotification(Icu_ChannelType Channel);

Table 3-13. Icu_EnableNotification Arguments

Type	Name	Direction	Description
Icu_ChannelType	Channel	input	Numeric identifier of the ICU channel

Return: void

This function is reentrant and enables the notification on the given channel.

pre:: Icu_Init must be called before.

3.5.12 Function Icu_GetInputState

This function returns the status of the ICU input.

Prototype: `Icu_InputStateType Icu_GetInputState(Icu_ChannelType Channel);`

Table 3-14. Icu_GetInputState Arguments

Type	Name	Direction	Description
Icu_ChannelType	Channel	input	Numeric identifier of the ICU channel

Return: Icu_InputStateType

Table 3-15. Icu_GetInputState Returns

Value	Description
ICU_ACTIVE	An activation edge has been detected
ICU_IDLE	No activation edge has been detected since the last call of Icu_GetInputState() or Icu_Init().

This service is reentrant shall return the status of the ICU input. Only channels which are configured for the following Icu_MeasurementMode shall be supported:

- ICU_MODE_SIGNAL_EDGE_DETECT,
- ICU_MODE_SIGNAL_MEASUREMENT.

pre:: Icu_Init must be called before.

3.5.13 Function Icu_StartTimestamp

This function starts the capturing of timer values on the edges.

Prototype: `void Icu_StartTimestamp(Icu_ChannelType Channel, Icu_ValueType *BufferPtr, uint16 BufferSize, uint16 NotifyInterval);`

Table 3-16. Icu_StartTimestamp Arguments

Type	Name	Direction	Description
Icu_ChannelType	Channel	input	Numeric identifier of the ICU channel

Table continues on the next page...

Table 3-16. Icu_StartTimestamp Arguments (continued)

Type	Name	Direction	Description
Icu_ValueType *	BufferPtr	input	Pointer to the buffer-array where the timestamp values shall be placed.
uint16	BufferSize	input	Size of the external buffer (number of entries)
uint16	NotifyInterval	input	Notification interval (number of events).

Return: void

This function is reentrant and starts the capturing of timer values on the edges activated by the service `Icu_SetActivationCondition()` to an external buffer.

pre:: Icu_Init must be called before.

3.5.14 Function Icu_StopTimestamp

This function stops the timestamp measurement of the given channel.

Prototype: void Icu_StopTimestamp(Icu_ChannelType Channel);

Table 3-17. Icu_StopTimestamp Arguments

Type	Name	Direction	Description
Icu_ChannelType	Channel	input	Numeric identifier of the ICU channel

Return: void

This function is reentrant and stops the timestamp measurement of the given channel.

pre:: Icu_Init must be called before.

3.5.15 Function Icu_GetTimestampIndex

This function reads the timestamp index of the given channel.

Prototype: Icu_IndexType Icu_GetTimestampIndex(Icu_ChannelType Channel);

Table 3-18. Icu_GetTimestampIndex Arguments

Type	Name	Direction	Description
Icu_ChannelType	Channel	input	Numeric identifier of the ICU channel

Return: Icu_IndexType - Timestamp index of the given channel+

This function is reentrant and reads the timestamp index of the given channel, which is next to be written.

pre:: Icu_Init must be called before. Icu_StartTimestamp must be called before.

3.5.16 Function Icu_ResetEdgeCount

This function resets the value of the counted edges to zero.

Prototype: void Icu_ResetEdgeCount(Icu_ChannelType Channel);

Table 3-19. Icu_ResetEdgeCount Arguments

Type	Name	Direction	Description
Icu_ChannelType	Channel	input	Numeric identifier of the ICU channel.

Return: void

This function is reentrant and resets the value of the counted edges to zero.

pre:: Icu_Init must be called before.

3.5.17 Function Icu_EnableEdgeCount

This function enables the counting of edges of the given channel.

Prototype: void Icu_EnableEdgeCount(Icu_ChannelType Channel);

Table 3-20. Icu_EnableEdgeCount Arguments

Type	Name	Direction	Description
Icu_ChannelType	Channel	input	Numeric identifier of the ICU channel

Return: void

This service is reentrant and shall enable the counting of edges of the given channel. Note: This service does not do the real counting itself. This is done by the hardware (capture unit). Only the configured edges shall be counted (rising edge / falling edge / both edges).

Configuration of the edge is done in Icu_Init or Icu_SetActivationCondition. The configured edge can be changed during runtime using Icu_SetActivationCondition. Interrupts are not required for edge counting. If interrupts are enabled, the interrupt service routine will set the overflow flag if more than 0xFFFF edges are measured.

pre:: Icu_Init must be called before. The given channel must be configured in Measurement Mode Edge Counter.

3.5.18 Function Icu_EnableEdgeDetection

This function enables / re-enables the detection of edges of the given channel.

Prototype: void Icu_EnableEdgeDetection(Icu_ChannelType Channel);

Table 3-21. Icu_EnableEdgeDetection Arguments

Type	Name	Direction	Description
Icu_ChannelType	Channel	input	Numeric identifier of the ICU channel

Return: void

This function is reentrant enables / re-enables the detection of edges of the given channel.

pre:: Icu_Init must be called before. The given channel must be configured in Measurement Mode Edge Counter.

3.5.19 Function Icu_DisableEdgeCount

This function disables the counting of edges of the given channel.

Prototype: void Icu_DisableEdgeCount(Icu_ChannelType Channel);

Table 3-22. Icu_DisableEdgeCount Arguments

Type	Name	Direction	Description
Icu_ChannelType	Channel	input	Numeric identifier of the ICU channel

Return: void

This function is reentrant and disables the counting of edges of the given channel.

pre:: Icu_Init must be called before. The given channel must be configured in Measurement Mode Edge Counter.

3.5.20 Function Icu_DisableEdgeDetection

This function disables the detection of edges of the given channel.

Prototype: `void Icu_DisableEdgeDetection(Icu_ChannelType Channel);`

Table 3-23. Icu_DisableEdgeDetection Arguments

Type	Name	Direction	Description
Icu_ChannelType	Channel	input	Numeric identifier of the ICU channel

Return: void

This function is reentrant and disables the detection of edges of the given channel.

pre:: Icu_Init must be called before. The given channel must be configured in Measurement Mode Edge Detection.

3.5.21 Function Icu_GetEdgeNumbers

This function reads the number of counted edges.

Prototype: `Icu_EdgeNumberType Icu_GetEdgeNumbers(Icu_ChannelType Channel);`

Table 3-24. Icu_GetEdgeNumbers Arguments

Type	Name	Direction	Description
Icu_ChannelType	Channel	input	Numeric identifier of the ICU channel

Return: Icu_EdgeNumberType - Number of the counted edges.

This function is reentrant reads the number of counted edges after the last call of `Icu_ResetEdgeCount()`.

pre:: Icu_Init must be called before. The given channel must be configured in Measurement Mode Edge Counter.

3.5.22 Function Icu_StartSignalMeasurement

This function starts the measurement of signals.

Prototype: `void Icu_StartSignalMeasurement(Icu_ChannelType Channel);`

Table 3-25. Icu_StartSignalMeasurement Arguments

Type	Name	Direction	Description
Icu_ChannelType	Channel	input	Numeric identifier of the ICU channel

Return: void

This service is reentrant and starts the measurement of signals beginning with the configured default start edge which occurs first after the call of this service. This service shall only be available in Measurement Mode ICU_MODE_SIGNAL_MEASUREMENT. This service shall reset the state for the given channel to ICU_IDLE.

pre:: Icu_Init must be called before. The given channel must be configured in Measurement Mode Signal Measurement.

3.5.23 Function Icu_StopSignalMeasurement

This function stops the measurement of signals of the given channel.

Prototype: void Icu_StopSignalMeasurement(Icu_ChannelType Channel);

Table 3-26. Icu_StopSignalMeasurement Arguments

Type	Name	Direction	Description
Icu_ChannelType	Channel	input	Numeric identifier of the ICU channel

Return: void

This function is reentrant and stops the measurement of signals of the given channel.

pre:: Icu_Init must be called before. The given channel must be configured in Measurement Mode Signal Measurement.

3.5.24 Function Icu_GetTimeElapsed

This function reads the elapsed Signal Low/High/Period Time for the given channel.

Prototype: Icu_ValueType Icu_GetTimeElapsed(Icu_ChannelType Channel);

Table 3-27. Icu_GetTimeElapsed Arguments

Type	Name	Direction	Description
Icu_ChannelType	Channel	input	Numeric identifier of the ICU channel

Return: Icu_ValueType - the elapsed Signal Low Time for the given channel that is configured in Measurement Mode Signal Measurement, Signal Low Time

This service is reentrant and reads the elapsed Signal Low Time for the given channel that is configured in Measurement Mode Signal Measurement, Signal Low Time. The elapsed time is measured between a falling edge and the consecutive rising edge of the channel. This service reads the elapsed Signal High Time for the given channel that is configured in Measurement Mode Signal Measurement, Signal High Time. The elapsed time is measured between a rising edge and the consecutive falling edge of the channel. This service reads the elapsed Signal Period Time for the given channel that is configured in Measurement Mode Signal Measurement, Signal Period Time. The elapsed time is measured between consecutive rising (or falling) edges of the channel. The period start edge is

configurable.

pre:: Icu_Init must be called before. The given channel must be configured in Measurement Mode Signal Measurement.

3.5.25 Function Icu_GetDutyCycleValues

This function reads the coherent active time and period time for the given ICU Channel.

Prototype: void Icu_GetDutyCycleValues(Icu_ChannelType Channel, Icu_DutyCycleType *DutyCycleValues);

Table 3-28. Icu_GetDutyCycleValues Arguments

Type	Name	Direction	Description
Icu_ChannelType	Channel	input	Numeric identifier of the ICU channel
Icu_DutyCycleType *	DutyCycleValues	output	Pointer to a buffer where the results (high time and period time) shall be placed.

Return: void

The function is reentrant and reads the coherent active time and period time for the given ICU Channel, if it is configured in Measurement Mode Signal Measurement, Duty Cycle Values.

pre:: Icu_Init must be called before. The given channel must be configured in Measurement Mode Signal Measurement, Duty Cycle Values.

3.5.26 Function Icu_GetVersionInfo

This service returns the version information of this module.

Prototype: `void Icu_GetVersionInfo(Std_VersionInfoType *versioninfo);`

Table 3-29. Icu_GetVersionInfo Arguments

Type	Name	Direction	Description
Std_VersionInfoType *	versioninfo	output	Pointer to location to store version info

Return: void

This service is Non reentrant and returns the version information of this module. The version information includes:

- Module Id
- Vendor Id
- Vendor specific version numbers If source code for caller and callee of this function is available this function should be realized as a macro. The macro should be defined in the modules header file.\

3.5.27 Function Icu_GetPulseWidth

This function returns the signals High time, Low time and Period without using the channel interrupt.

Prototype: `void Icu_GetPulseWidth(Icu_channelType Channel);`

Table 3-30. Icu_GetPulseWidth Arguments

Type	Name	Direction	Description
Icu_ChannelType	Channel	input	- Numeric identifier of the ICU channel

This function is reentrant and returns the signals High time, Low time and Period without using the channel interrupt.

pre.: Icu_Init must be called before.

3.5.28 Function Icu_LLD_StartSignalMeasurement

Icu driver function that starts the signal measurement of ICU channel.

Prototype: void Icu_LLD_StartSignalMeasurement(const Icu_ChannelConfigType *tempPtr, const Icu_ChannelType hwChannel, Icu_ParamType tempProperty, const Icu_ChannelType msChannel, Icu_ChannelType numChl);

Table 3-31. Icu_LLD_StartSignalMeasurement Arguments

Type	Name	Direction	Description
const Icu_ChannelConfigType *	tempPtr	input	- Buffer pointer
const Icu_ChannelType	hwChannel	input	- ICU hardware channel number
Icu_ParamType	tempProperty	input	- Time measurement type (HIGH or LOW)
const Icu_ChannelType	msChannel	input	- The index of ICU logical channel
Icu_ChannelType	numChl	input	- The index of ICU channel for current configuration structure

This function:

- Configures the Activation type depends on tempProperty value
- starts the signal measurement of ICU channel.

3.6 Structure Definitions

Data structures supported by the driver are as per AUTOSAR ICU Driver software specification Version 4.0 Rev0003.

3.6.1 Structure Icu_ConfigType

This type contains initialization data.

he notification functions shall be configurable as function pointers within the initialization data structure (Icu_ConfigType). This type of the external data structure shall contain the initialization data for the ICU driver. It shall contain:

- Wakeup Module Info (in case the wakeup-capability is true)
- ICU dependent properties for used HW units
- Clock source with optional prescaler (if provided by HW)

Declaration

```
typedef struct
{
    const Icu_ChannelConfigType * Icu_ChannelConfigPtr,
```

Structure Definitions

```
const Icu_ChannelType * Icu_ChannelId,
const Icu_ChannelType * Icu_ChannelIndex,
const Icu_ChannelType * Icu_HWMap,
const Icu_ChannelType * Icu_HWWKMap,
const Icu_ClockPrescalerType Icu_Irq_AlternateClockPrescaler,
const Icu_ClockPrescalerType Icu_Irq_ClockPrescaler,
const Icu_ChannelType Icu_MaxChannels
} Icu_ConfigType;
```

Table 3-32. Structure Icu_ConfigType member description

Member	Description
Icu_ChannelConfigPtr	This index relates the Icu Channel number with the respective global variable, depending on the measurement mode Each kind of measurement mode has an array(s) in the ICU driver..
Icu_ChannelId	This index relates the Hardware channels with the respective ICU channel..
Icu_ChannelIndex	Array for mapping the Icu channels to hardware channels.
Icu_HWMap	This index relates the Hardware channels with the respective ICU channel
Icu_HWWKMap	This index relates the Wakeup pins with the respective ICU channel.
Icu_Irq_AlternateClockPrescaler	This is alternate value of prescaler used for clock
Icu_Irq_ClockPrescaler	This is alue of prescaler used for clock.
Icu_MaxChannels	The number of configured channels. <Pointer to Icu channel configuration.

3.6.2 Structure Icu_ChannelConfigType

Structure that contains ICU channel configuration.

It contains the information like Icu Channel Mode, Channel Notification function, overflow Notification function.

Declaration

```
typedef struct
{
    Icu_MeasurementModeType Icu_Channel_Mode,
    Icu_NotifyType Icu_Channel_Notification,
    Icu_NotifyType Icu_Channel_OverFlowNotification,
    Icu_MeasurementSubModeType Icu_Channel_Property,
    Icu_WakeupValue Icu_Channel_WakeupValue,
    Icu_ParamType Icu_ParamValue,
    boolean IcuDisableEcumWakeupSourceNotification,
    boolean IcuSignalMeasureWithoutInterrupt
} Icu_ChannelConfigType;
```

Table 3-33. Structure Icu_ChannelConfigType member description

Member	Description
Icu_Channel_Mode	EDGE_DETECT, TIME_STAMP, SIGNAL_MEASUREMENT or EDGE_COUNTER.

Table continues on the next page...

Table 3-33. Structure Icu_ChannelConfigType member description (continued)

Member	Description
Icu_Channel_Notification	Icu Channel Notification function for TIME_STAMP or EDGE_COUNTER mode.
Icu_Channel_OverflowNotification	Icu Channel Overflow Notification function.
Icu_Channel_Property	CIRCULAR_BUFFER or LINEAR_BUFFER for TIME_STAMP, DUTY_CYCLE, HIGH_TIME, LOW_TIME or PERIOD_TIME for SIGNAL_MEASUREMENT and RISING_EDGE, FALLING_EDGE or BOTH_EDGES for EDGE_COUNTER.
Icu_Channel_WakeupValue	EcuM wakeup source Id.
Icu_ParamValue	Configuration parameters of hardware channel registers.

3.6.3 Structure Icu_DutyCycleType

Structure that contains ICU Duty cycle parameters. It contains the values needed for calculating duty cycles i.e Period time value and active time value.

Declaration

```
typedef struct
{
    Icu_ValueType ActiveTime,
    Icu_ValueType PeriodTime
} Icu_DutyCycleType;
```

Table 3-34. Structure Icu_DutyCycleType member description

Member	Description
ActiveTime	Low or High time value.
PeriodTime	Period time value.

3.6.4 Structure Std_VersionInfoType

This type shall be used to request the version of a BSW module using the "ModuleName"_GetVersionInfo() function.

Declaration

```
typedef struct
{
    uint16 moduleID,
    uint8 sw_major_version,
    uint8 sw_minor_version,
    uint8 sw_patch_version,
    uint16 vendorID
} Std_VersionInfoType;
```

Table 3-35. Structure Std_VersionInfoType member description

Member	Description
moduleID	BSW module ID.
sw_major_version	BSW module software major version.
sw_minor_version	BSW module software minor version.
sw_patch_version	BSW module software patch version.
vendorID	vendor ID

3.7 Enum Definitions

Data Enums supported by the driver are as per AUTOSAR ICU Driver software specification Version 4.0 Rev0003.

3.7.1 Enumeration Icu_SelectPrescalerType

Definition of prescaler type.

Enumeration of available prescalers.

Table 3-36. Enumeration Icu_SelectPrescalerType Values

Value	Description
ICU_NORMAL = 0x0U	Default channel prescaler option
ICU_ALTERNATE = 0x1U	Alternative prescaler to be used depending on user application (e.g. low power modes)

3.7.2 Enumeration Icu_LevelType

Return the status of the pin.

Enumeration of to check the status of emios pin.

Table 3-37. Enumeration Icu_LevelType Values

Value	Description
ICU_LEVEL_LOW = 0x0U	Logical level LOW of pin
ICU_LEVEL_HIGH = 0x1U	Logical level HIGH of pin

3.7.3 Enumeration Icu_ActivationType

Definition of the type of activation of an ICU channel.

Table 3-38. Enumeration Icu_ActivationType Values

Value	Description
ICU_FALLING_EDGE = 0x0U	An appropriate action shall be executed when a falling edge occurs on the ICU input signal.
ICU_RISING_EDGE = 0x1U	An appropriate action shall be executed when a rising edge occurs on the ICU input signal.
ICU_BOTH_EDGES = 0x2U	An appropriate action shall be executed when either a rising or falling edge occur on the ICU input signal.

3.7.4 Enumeration Icu_InputStateType

Input state of an ICU channel.

Table 3-39. Enumeration Icu_InputStateType Values

Value	Description
ICU_ACTIVE = 0U	An activation edge has been detected.
ICU_IDLE	No activation edge has been detected since the last call of Icu_GetInputState() or Icu_Init().

3.7.5 Enumeration Icu_MeasurementModeType

Definition of the measurement mode type.

Table 3-40. Enumeration Icu_MeasurementModeType Values

Value	Description
ICU_MODE_SIGNAL_EDGE_DETECT = 0U	Mode for detecting edges.
ICU_MODE_SIGNAL_MEASUREMENT	Mode for measuring different times between various configurable edges.
ICU_MODE_TIMESTAMP	Mode for capturing timer values on configurable edges.
ICU_MODE_EDGE_COUNTER	Mode for counting edges on configurable edges.

3.7.6 Enumeration Icu_ModeType

Allow enabling / disabling of all interrupts which are not required for the ECU wakeup.

Table 3-41. Enumeration Icu_ModeType Values

Value	Description
ICU_MODE_NORMAL = 0U	Normal operation, all used interrupts are enabled according to the notification requests.
ICU_MODE_SLEEP	Reduced power operation. In sleep mode only those notifications are available which are configured as wakeup capable.

3.7.7 Enumeration Icu_SignalMeasurementPropertyType

Definition of the measurement property type.

Table 3-42. Enumeration Icu_SignalMeasurementPropertyType Values

Value	Description
ICU_LOW_TIME = 0U	The channel is configured for reading the elapsed Signal Low Time.
ICU_HIGH_TIME	The channel is configured for reading the elapsed Signal High Time.
ICU_PERIOD_TIME	The channel is configured for reading the elapsed Signal Period Time.
ICU_DUTY_CYCLE	he channel is configured to read values which are needed for calculating the duty cycle (coherent Active and Period Time).

3.7.8 Enumeration Icu_TimestampBufferType

Definition of the timestamp measurement property type.

Table 3-43. Enumeration Icu_TimestampBufferType Values

Value	Description
ICU_LINEAR_BUFFER = 0U	The buffer will just be filled once.
ICU_CIRCULAR_BUFFER	After reaching the end of the buffer, the driver restarts at the beginning of the buffer.

3.7.9 Enumeration Icu_WakeupCapableType

Definition of the type of wake up capability of an ICU channel.

Table 3-44. Enumeration Icu_WakeupCapableType Values

Value	Description
ICU_WAKEUP_NOTCAPABLE = 0x0U	Channel is not wakeup capable.
ICU_WAKEUP_CAPABLE = 0x1U	TChannel is wakeup capable.

3.8 Typedef Definitions

Data Typedefs supported by the driver are as per AUTOSAR ICU Driver software specification Version 4.0 Rev0003.

3.8.1 Typedef Icu_EdgeNumberType

Type for saving hardware specific edge number.

Type: Icu_HwSpecificEdgeNumberType

3.8.2 Typedef Icu_ChannelType

This gives the numeric ID (hardware channel number) of an ICU channel.

Type: uint8

3.8.3 Typedef Icu_ChannelStateType

ICU Channel state type.

Type: uint8

3.8.4 Typedef Icu_IndexType

Type for saving the ICU Hardware specific index.

Type: Icu_HwSpecificIndexType

3.8.5 Typedef Icu_MeasurementSubModeType

Type for saving the ICU measurement submode type.

Type: uint8

3.8.6 Typedef Icu_NotifyType

The notification functions shall have no parameters and no return value.

Type: void (*Icu_NotifyType)(void)

3.8.7 Typedef Icu_ValueType

Type for saving the timer register width value.

Type: Icu_TimerRegisterWidthType

3.8.8 Typedef Icu_ParamType

The Icu_ParamType contains combined bit fields for initialization options, for different registers.

Type: uint32

3.9 Define Definitions

3.9.1 Define ICU_CHECKWAKEUP_ID

API service ID for Icu_CheckWakeup function.

Definition: #define ICU_CHECKWAKEUP_ID 0x15U

3.9.2 Define ICU_DEINIT_ID

API service ID for Icu_DeInit function.

Definition: `#define ICU_DEINIT_ID 0x01U`

3.9.3 Define ICU_DISABLEEDGECOUNT_ID

API service ID for Icu_DisableEdgeCount function.

Definition: `#define ICU_DISABLEEDGECOUNT_ID 0x0EU`

3.9.4 Define ICU_DISABLEEDGEDETECTION_ID

API service ID for Icu_DisableEdgeDetection function.

Definition: `#define ICU_DISABLEEDGEDETECTION_ID 0x17U`

3.9.5 Define ICU_DISABLENOTIFICATION_ID

API service ID for Icu_DisableNotification function.

Definition: `#define ICU_DISABLENOTIFICATION_ID 0x06U`

3.9.6 Define ICU_DISABLEWAKEUP_ID

API service ID for Icu_DisableWakeup function.

Definition: `#define ICU_DISABLEWAKEUP_ID 0x03U`

3.9.7 Define ICU_ENABLEEDGECOUNT_ID

API service ID for Icu_EnableEdgeCount function.

Definition: `#define ICU_ENABLEEDGECOUNT_ID 0x0DU`

3.9.8 Define ICU_ENABLEEDGEDETECTION_ID

API service ID for Icu_EnableEdgeDetection function.

Definition: `#define ICU_ENABLEEDGEDETECTION_ID 0x16U`

3.9.9 Define ICU_ENABLENOTIFICATION_ID

API service ID for Icu_EnableNotification function.

Definition: `#define ICU_ENABLENOTIFICATION_ID 0x07U`

3.9.10 Define ICU_ENABLEWAKEUP_ID

API service ID for Icu_EnableWakeup function.

Definition: `#define ICU_ENABLEWAKEUP_ID 0x04U`

3.9.11 Define ICU_GETDUTYCYCLEVALUES_ID

API service ID for Icu_GetDutyCycleValues function.

Definition: `#define ICU_GETDUTYCYCLEVALUES_ID 0x11U`

3.9.12 Define ICU_GETEDGENUMBERS_ID

API service ID for Icu_GetEdgeNumbers function.

Definition: `#define ICU_GETEDGENUMBERS_ID 0x0FU`

3.9.13 Define ICU_GETINPUTSTATE_ID

API service ID for Icu_GetInputState function.

Definition: `#define ICU_GETINPUTSTATE_ID 0x08U`

3.9.14 Define ICU_GETTIMEELAPSED_ID

API service ID for Icu_GetTimeElapsed function.

Definition: `#define ICU_GETTIMEELAPSED_ID 0x10U`

3.9.15 Define ICU_GETTIMESTAMPINDEX_ID

API service ID for Icu_GetTimestampIndex function.

Definition: `#define ICU_GETTIMESTAMPINDEX_ID 0x0BU`

3.9.16 Define ICU_GETVERSIONINFO_ID

API service ID for Icu_GetVersionInfo function.

Definition: `#define ICU_GETVERSIONINFO_ID 0x12U`

3.9.17 Define ICU_INIT_ID

API service ID for Icu_Init function.

Definition: `#define ICU_INIT_ID 0x00U`

3.9.18 Define ICU_RESETEGECOUNT_ID

API service ID for Icu_ResetEdgeCount function.

Definition: `#define ICU_RESETEGECOUNT_ID 0x0CU`

3.9.19 Define ICU_SET_CLOCK_MODE_ID

API service ID for Icu_SetClockMode function.

Definition: `#define ICU_SET_CLOCK_MODE_ID 0x7BU`

3.9.20 Define ICU_GET_INPUT_LEVEL_ID

API service ID for Icu_GetInputLevel function.

Definition: `#define ICU_GET_INPUT_LEVEL_ID 0x7CU`

3.9.21 Define ICU_SETACTIVATIONCONDITION_ID

API service ID for Icu_SetActivationCondition function.

Definition: `#define ICU_SETACTIVATIONCONDITION_ID 0x05U`

3.9.22 Define ICU_SETMODE_ID

API service ID for Icu_SetMode function.

Definition: `#define ICU_SETMODE_ID 0x02U`

3.9.23 Define ICU_STARTSIGNALMEASUREMENT_ID

API service ID for Icu_StartSignalMeasurement function.

Definition: `#define ICU_STARTSIGNALMEASUREMENT_ID 0x13U`

3.9.24 Define ICU_STARTTIMESTAMP_ID

API service ID for Icu_StartTimestamp function.

Definition: `#define ICU_STARTTIMESTAMP_ID 0x09U`

3.9.25 Define ICU_STOPSIGNALMEASUREMENT_ID

API service ID for Icu_StopSignalMeasurement function.

Definition: `#define ICU_STOPSIGNALMEASUREMENT_ID 0x14U`

3.9.26 Define ICU_STOPTIMESTAMP_ID

API service ID for Icu_StopTimestamp function.

Definition: `#define ICU_STOPTIMESTAMP_ID 0x0AU`

3.9.27 Define ICU_E_ALREADY_INITIALIZED

API Icu_Init service called when the ICU driver and the Hardware are already initialized.

Definition: `#define ICU_E_ALREADY_INITIALIZED 0x17U`

3.9.28 Define ICU_E_BUSY_OPERATION

API service Icu_SetMode is called while a running operation.

Definition: `#define ICU_E_BUSY_OPERATION 0x16U`

3.9.29 Define ICU_E_EDGECOUNTING_OVERFLOW

API Icu_GetEdgeNumbers service called when the Counter rolls over.

Definition: `#define ICU_E_EDGECOUNTING_OVERFLOW 0xD0U`

3.9.30 Define ICU_E_MEASUREMENT_OVERFLOW

API Icu_GetTimeElapsed service called when the Time elapsed overflows.

Definition: `#define ICU_E_MEASUREMENT_OVERFLOW 0xD2U`

3.9.31 Define ICU_E_NOT_STARTED

API service Icu_StopTimestamp called on a channel which was not started or already stopped.

Definition: `#define ICU_E_NOT_STARTED 0x15U`

3.9.32 Define ICU_E_PARAM_ACTIVATION

API service used with an invalid or not feasible activation.

Definition: `#define ICU_E_PARAM_ACTIVATION 0x0CU`

3.9.33 Define ICU_E_PARAM_BUFFER_PTR

API service used with an invalid application-buffer pointer.

Definition: `#define ICU_E_PARAM_BUFFER_PTR 0x0DU`

3.9.34 Define ICU_E_PARAM_BUFFER_SIZE

API service used with an invalid buffer size.

Definition: `#define ICU_E_PARAM_BUFFER_SIZE 0x0EU`

3.9.35 Define ICU_E_PARAM_CHANNEL

API service used with an invalid channel identifier or channel was not configured for the functionality of the calling API.

Definition: `#define ICU_E_PARAM_CHANNEL 0x0BU`

3.9.36 Define ICU_E_PARAM_CLOCK_MODE

API Icu_SetClockMode service called with wrong parameter.

Definition: `#define ICU_E_PARAM_CLOCK_MODE 0x7AU`

3.9.37 Define ICU_E_PARAM_NOTIFY_INTERVAL

API Icu_StartTimeStamp is called and the parameter NotifyInterval is invalid (e.g. 0, NotifyInterval < 1)

Definition: `#define ICU_E_PARAM_NOTIFY_INTERVAL 0x18U`

3.9.38 Define ICU_E_PARAM_VINFO

API Icu_GetVersionInfo is called and the parameter versioninfo is invalid (e.g. NULL)

Definition: `#define ICU_E_PARAM_VINFO 0x19U`

3.9.39 Define ICU_E_PARAM_CONFIG

API Icu_Init service called with wrong parameter.

Definition: `#define ICU_E_PARAM_CONFIG 0x0AU`

3.9.40 Define ICU_E_PARAM_MODE

API service Icu_SetMode used with an invalid mode.

Definition: `#define ICU_E_PARAM_MODE 0x0FU`

3.9.41 Define ICU_E_TIMESTAMP_OVERFLOW

API Icu_GetTimestampIndex service called when the Time stamp count overflows.

Definition: `#define ICU_E_TIMESTAMP_OVERFLOW 0xD1U`

3.9.42 Define ICU_CAPTUREREGISTER_ID

API service ID for Icu_GetCaptureRegisterValue function.

Definition: `#define ICU_CAPTUREREGISTER_ID 0x7EU`

3.9.43 Define ICU_E_UNINIT

API service used without module initialization.

Definition: `#define ICU_E_UNINIT 0x14U`

3.10 Configuration Parameters

As per the AUTOSAR specification the driver has two types of configurations parameters: **Pre-Compile** parameters and **Post-Build** parameters.

These can be selected using “IMPLEMENTATION_CONFIG_VARIANT”

Pre-Compile parameters are stored in the file "Icu_Cfg.h" and "Icu_Cfg.c". Post-Build parameters are stored in the file "Icu_PBcfg.c".

The files to be used for different configuration types are listed below:

1. **Variant PC:** Icu_Cfg.h, Icu_Cfg.c
2. **Variant PB:** Icu_Cfg.h, Icu_PBcfg.c
3. **Variant LT:** Not Applicable

Configuration Parameters

A section for **Icu_PBcfg.c** file is needed in linker file to place the post build configuration in desired location. Please refer to section-8 of AUTOSAR_SWS_C_ImplementationRules file for complete details on configuration types.

3.10.1 Plugin interface

The following picture gives an example of configuration screen for all ICU parameters in Tresos® Studio configuration tool.

The screenshot displays the configuration interface for the Icu plugin in Tresos Studio. The main window is titled "Icu" and contains a "Name" field with the value "Icu". Below this, there are three tabs: "General", "IcuConfigSet", and "Published Information". The "General" tab is active, showing a tree view with three expandable sections: "IcuGeneral", "IcuNonAUTOSAR", and "IcuOptionalApis".

- IcuGeneral**
 - Name: IcuGeneral
 - IcuDevErrorDetect: ☒ (with icons for help, status, and edit)
 - IcuIndex (0 -> 255): 0 (with a help icon)
 - IcuReportWakeupSource: ☒ (with icons for help, status, and edit)
- IcuNonAUTOSAR**
 - Name: IcuNonAUTOSAR
 - IcuEnableDualClockMode: ☐ (with icons for help, status, and edit)
 - IcuOverflowNotificationApi: ☐ (with icons for help, status, and edit)
 - IcuAllowWkpOnAllChls: ☐ (with icons for help, status, and edit)
 - IcuGetCaptureRegisterValueApi: ☒ (with icons for help, status, and edit)
 - IcuGetInputLevelApi: ☒ (with icons for help, status, and edit)
- IcuOptionalApis**
 - Name: IcuOptionalApis
 - IcuDeInitApi: ☒ (with icons for help, status, and edit)
 - IcuEdgeCountApi: ☒ (with icons for help, status, and edit)
 - IcuGetDutyCycleValuesApi: ☒ (with icons for help, status, and edit)
 - IcuGetTimeElapsedApi: ☒ (with icons for help, status, and edit)
 - IcuSetModeApi: ☒ (with icons for help, status, and edit)
 - IcuTimestampApi: ☒ (with icons for help, status, and edit)
 - IcuDisableWakeupApi: ☒ (with icons for help, status, and edit)
 - IcuEnableWakeupApi: ☒ (with icons for help, status, and edit)
 - IcuGetInputStateApi: ☒ (with icons for help, status, and edit)
 - IcuGetVersionInfoApi: ☒ (with icons for help, status, and edit)
 - IcuSignalMeasurementApi: ☒ (with icons for help, status, and edit)
 - IcuWakeupFunctionalityApi: ☒ (with icons for help, status, and edit)

Figure 3-2. Tresos Plugin snapshot for IcuGeneral form

NOTE

If pre-compile variant is preferred,

- IMPLEMENTATION_CONFIG_VARIANT in Tresos GUI should be selected as “VariantPreCompile” as shown below

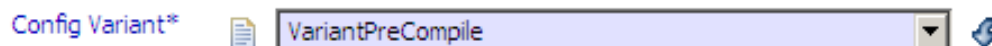


Figure 3-3. Tresos Plugin snapshot for IcuConfigurationVariant form

The following picture gives an example of configuration screen for all ICU configuration set.

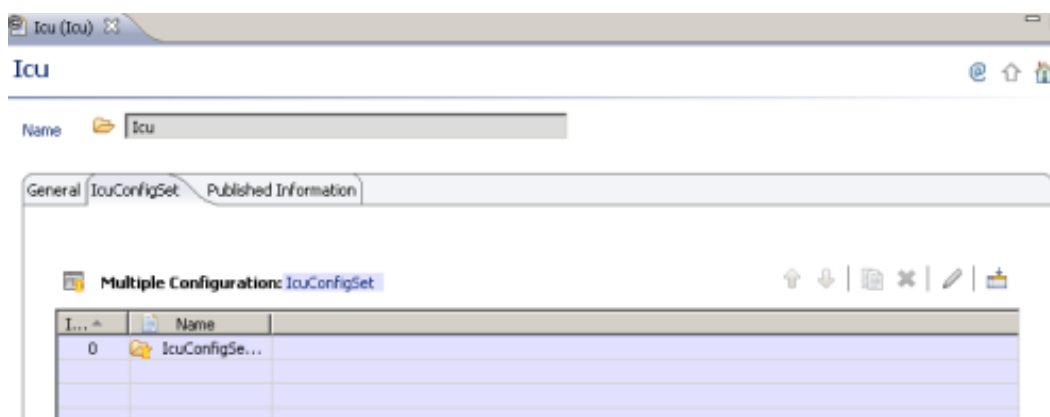


Figure 3-4. Tresos Plugin snapshot for IcuConfigsSet form

The following picture gives an example of configuration screen for General configuration for a specific set

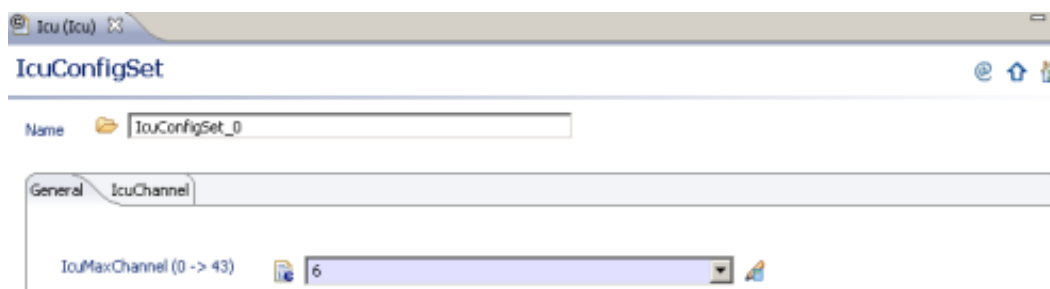


Figure 3-5. Tresos Plugin snapshot for Icu_MaxChannel form for a specific set

The following picture gives an example for IcuChannel container

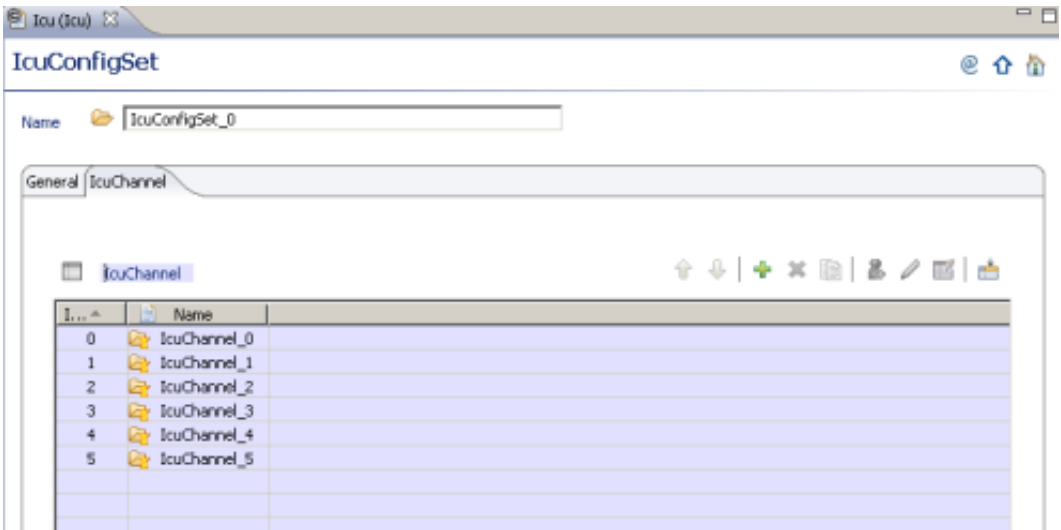


Figure 3-6. Tresos Plugin snapshot for Icu_Channel container

The following picture gives an example for a configuration screen of an Icu channel

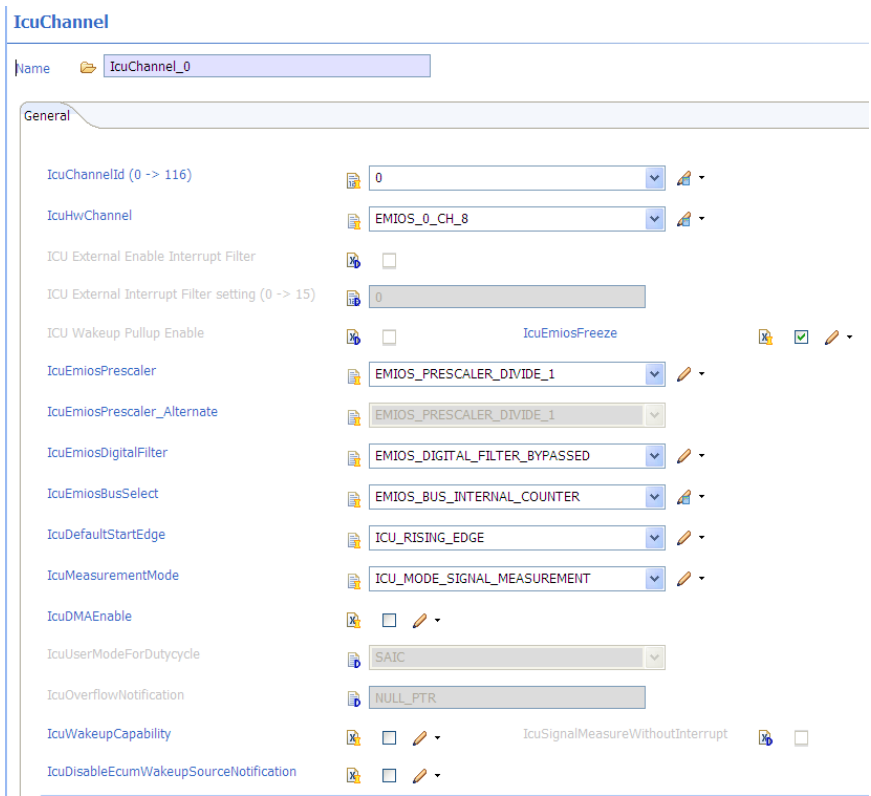


Figure 3-7. Tresos Plugin snapshot for Icu_Channel form

3.10.2 Pre-Compile Parameters

Pre-Compile parameters, their possible values and meaning are described in the following text. Pre-Compile parameters are implemented as preprocessor defines.

NOTE

If pre-compile variant is preferred, then:

- IMPLEMENTATION_CONFIG_VARIANT in Tresos GUI should be selected as “VariantPreCompile” as shown below

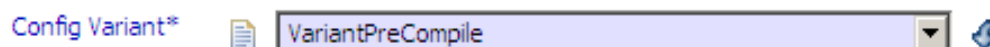


Figure 3-8. Tresos Plugin snapshot for IcuConfigurationVariant form

- Icu_Cfg.c should be compiled
- Icu_PBcfg.c should not be compiled

3.10.2.1 icuallowwkponallchls

Table 3-45. icuallowwkponallchls

Description	This switch allows wakeup capability for all ICU channels. true: Enabled. false: Disabled.
Class	Non-autosar Parameter
Range	True,false
Default	false
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ChannelConfigType, ICU_CONST) Icu_InitPBChannel_0[26]= { { (((Icu_ParamType)ICU_WAKEUP_CAPABLE << ICU_WAKEUP_SHIFT) (ICU_RISING_EDGE << ICU_EDGE_PARAM_SHIFT) (EMIOS_FREEZE_ENABLE << ICU_EMIOS_FREEZE_PARAM_SHIFT) (EMIOS_PRESCALER_DIVIDE_1 << ICU_EMIOS_PRESCALER_PARAM_SHIFT) (EMIOS_DIGITAL_FILTER_BYPASSED << ICU_EMIOS_DIGITAL_FILTER_PARAM_SHIFT) (EMIOS_BUS_A << ICU_EMIOS_BUS_SELECT_PARAM_SHIFT)), ICU_MODE_SIGNAL_EDGE_DETECT, 0U, Icu_Channel_1_Notification, 0U }, </pre>

Table 3-45. icuallowwkponallchls

	...
	}

3.10.2.2 IcuChannelId

Table 3-46. IcuChannelId

Description	Channel Id of the ICU channel.
Class	Autosar Parameter
Range	0-40
Default	0
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ConfigType, ICU_CONST) IcuConfigSet_0 = { 1, /* The number of channels configured*/ Icu_InitPBChannel_0, Icu_InitPBChannelIndex_0, Icu_ChannelId_0, Icu_InitPBHWMap_0, Icu_InitPBHWKMap_0 }; </pre>

3.10.2.3 IcuDelInitApi

Table 3-47. IcuDelInitApi

Description	Adds / removes the service Icu_DelInit() from the code
Class	Autosar Parameter
Range	True,false
Default	True
Source File	Icu_Cfg.h
Source Representation	<code>#define ICU_DE_INIT_API <STD_OFF, STD_ON></code>

3.10.2.4 IcuDevErrorDetect

Table 3-48. IcuDevErrorDetect

Description	Switches the Development Error Detection and Notification ON or OFF
Class	Autosar Parameter

Table continues on the next page...

Table 3-48. IcuDevErrorDetect (continued)

Range	True,false
Default	True
Source File	Icu_Cfg.h
Source Representation	#define ICU_DEV_ERROR_DETECT <STD_OFF, STD_ON>

3.10.2.5 IcuDisableWakeupApi

Table 3-49. IcuDisableWakeupApi

Description	Adds / removes the service Icu_DisableWakeup () from the code.
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Icu_Cfg.h
Source Representation	#define ICU_DISABLE_WAKEUP_API<STD_OFF, STD_ON>

3.10.2.6 IcuDmaMaxChannel

Table 3-50. IcuDmaMaxChannel

Description	This parameter contains the number of Channels configured with DMA enabled.
Class	Autosar Parameter
Range	0 to 31
Default	0
Source File	Icu_Cfg.h
Source Representation	#define ICU_MAX_DMA_CHANNEL 6U

3.10.2.7 IcuEdgeCountApi

Table 3-51. IcuEdgeCountApi

Description	Adds / removes all services related to the edge counting functionality as listed below, from the code: Icu_ResetEdgeCount(), Icu_EnableEdgeCount(), Icu_DisableEdgeCount(), Icu_GetEdgeNumbers()
Class	Autosar Parameter
Range	True , False
Default	True

Table continues on the next page...

Table 3-51. IcuEdgeCountApi (continued)

Source File	Icu_Cfg.h
Source Representation	#define ICU_EDGE_COUNT_API <STD_OFF, STD_ON>

3.10.2.8 IcuEdgeDetectApi

Table 3-52. IcuEdgeDetectApi

Description	Adds / removes the services Icu_EnableEdgeDetection() and Icu_DisableEdgeDetection() from the code
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Icu_Cfg.h
Source Representation	#define ICU_EDGE_DETECT_API <STD_OFF, STD_ON>

3.10.2.9 IcuEnableDualClockMode

Table 3-53. IcuEnableDualClockMode

Description	Adds / removes the service Icu_SetClockMode () from the code.
Class	Non Autosar Parameter
Range	True,false
Default	False
Source File	Icu_Cfg.h
Source Representation	#define ICU_DUAL_CLOCK_MODE <STD_OFF, STD_ON>

3.10.2.10 IcuEnableWakeupApi

Table 3-54. IcuEnableWakeupApi

Description	Adds / removes the service Icu_EnableWakeup () from the code.
Class	Autosar Parameter
Range	True,false
Default	True
Source File	Icu_Cfg.h
Source Representation	#define ICU_ENABLE_WAKEUP_API <STD_OFF, STD_ON>

3.10.2.11 IcuEXT_ISR_IFCPRDigitalFilter

Table 3-55. IcuEXT_ISR_IFCPRDigitalFilter

Description	Configures a clock prescaler used to select the clock for all digital filter counters in SIUL.
Class	Autosar Parameter
Range	0 to 15
Default	0
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ConfigType, ICU_CONST) IcuConfigSet_0 = { (Icu_ChannelType)16U, /* The number of channels configured*/ (Icu_ClockPrescalerType)10U, Icu_InitPBChannel_0, Icu_InitPBChannelIndex_0, Icu_InitPBHWMap_0, Icu_InitPBHWKMap_0 }; </pre>

3.10.2.12 IcuEXT_ISRaltIFCPRDigitalFilter

Table 3-56. IcuEXT_ISRaltIFCPRDigitalFilter

Description	This parameter configures the alternate clock divider value for the internal prescaler of specific Unified Channel. Prescaled clock used as clock source for the programmable input filter.
Class	Autosar Parameter
Range	0-15
Default	0
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ConfigType, ICU_CONST) Icu_Configs_PC = { (Icu_ChannelType)2, /* The number of channels configured*/ Icu_InitChannel_PC, Icu_InitChannelIndex_PC, Icu_ChannelId_PC, Icu_InitHWMap_PC, Icu_InitHWKMap_PC, Icu_DMA_InitHWMap_PC, Icu_InitDMAChannelIndex_PC, (Icu_ClockPrescalerType)0U, #if (ICU_DUAL_CLOCK_MODE == STD_ON) (Icu_ClockPrescalerType)0U, #endif (Icu_WakeupValue)0xffffffffU }; </pre>

3.10.2.13 IcuGetDutyCycleValuesApi

Table 3-57. IcuGetDutyCycleValuesApi

Description	Adds / removes the service Icu_GetDutyCycleValues () from the code. If IcuSignalMeasurementApi == OFF this switch should be set to OFF
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Icu_Cfg.h
Source Representation	<code>#define ICU_GET_DUTY_CYCLE_VALUES_API <STD_OFF, STD_ON></code>

3.10.2.14 IcuGetInputLevelApi

Table 3-58. IcuGetInputLevelApi

Description	Adds / removes the service Icu_GetInputLevel() from the code
Class	Autosar Parameter
Range	ICU_LEVEL_HIGH,ICU_LEVEL_LOW
Default	ICU_LEVEL_LOW
Source File	Icu_Cfg.h
Source Representation	<code>#define ICU_GET_INPUT_LEVEL_API <STD_OFF, STD_ON></code>

3.10.2.15 IcuGetInputStateApi

Table 3-59. IcuGetInputStateApi

Description	Adds / removes the service Icu_GetInputState () from the code.
Class	Autosar Parameter
Range	True,false
Default	True
Source File	Icu_Cfg.h
Source Representation	<code>#define ICU_GET_INPUT_STATE_API <STD_OFF, STD_ON></code>

3.10.2.16 IcuGetTimeElapsedApi

Table 3-60. IcuGetTimeElapsedApi

Description	Adds / removes the service Icu_GetTimeElapsed () from the code. If IcuSignalMeasurementApi == OFF this switch should be set to OFF
Class	Autosar Parameter
Range	True,false
Default	True
Source File	Icu_Cfg.h
Source Representation	<code>#define ICU_GET_TIME_ELAPSED_API <STD_OFF, STD_ON></code>

3.10.2.17 IcuGetVersionInfoApi

Table 3-61. IcuGetVersionInfoApi

Description	Adds / removes the service Icu_GetVersionInfo () from the code.
Class	Autosar Parameter
Range	True,false
Default	True
Source File	Icu_Cfg.h
Source Representation	<code>#define ICU_GET_VERSION_INFO_API <STD_OFF, STD_ON></code>

3.10.2.18 IcuHwChannel

Table 3-62. IcuHwChannel

Description	Channel Id of the ICU channel.
Class	Autosar Parameter
Range	EMIOS_0_CH[0-31], EMIOS_1_CH[0-31],IRQ[0-23],WKUP[0-31]
Default	EMIOS_0_CH_0
Source File	Icu_Cfg.h
Source Representation	The highlighted portion of the following structure: <pre>#define ICU_HW_CHANNELS { \ ICU_IRQ_0, \ ICU_WKUP_4, \ ICU_IRQ_2, \ ICU_EMIOS_0_CH_9, \ ICU_EMIOS_0_CH_8, \ ICU_EMIOS_1_CH_16, \</pre>

Table 3-62. IcuHwChannel

	<i>ICU_EMIOS_1_CH_23, \</i> <i>ICU_EMIOS_0_CH_16, \</i> <i>ICU_EMIOS_0_CH_10, \</i> <i>ICU_EMIOS_0_CH_11, \</i> <i>ICU_EMIOS_0_CH_2, \</i> <i>ICU_EMIOS_0_CH_3, \</i> <i>ICU_EMIOS_1_CH_22</i> <i>\}</i>
--	---

Table 3-63. Note:The following channels support IPM and IPWM mode .

ICU channels(EMIOS_A,EMIOS_B)- With IPM,IPWM mode:	ICU channels(EMIOS_A)- Without IPM,IPWM mode:	ICU channels(EMIOS_B)- Without IPM,IPWM mode:
<i>{ \</i> <i>EMIOS_0_CH_1, \</i> <i>EMIOS_0_CH_2, \</i> <i>EMIOS_0_CH_3, \</i> <i>EMIOS_0_CH_4, \</i> <i>EMIOS_0_CH_5, \</i> <i>EMIOS_0_CH_6, \</i> <i>EMIOS_0_CH_7, \</i> <i>EMIOS_0_CH_9, \</i> <i>EMIOS_0_CH_10, \</i> <i>EMIOS_0_CH_11, \</i> <i>EMIOS_0_CH_12, \</i> <i>EMIOS_0_CH_13, \</i> <i>EMIOS_0_CH_14, \</i> <i>EMIOS_0_CH_15, \</i> <i>EMIOS_1_CH_9, \</i> <i>EMIOS_1_CH_11, \</i> <i>EMIOS_1_CH_12, \</i> <i>EMIOS_1_CH_13, \</i> <i>EMIOS_1_CH_14, \</i> <i>EMIOS_1_CH_15</i> <i>\}</i>	<i>{ \</i> <i>EMIOS_0_CH_0, \</i> <i>EMIOS_0_CH_8, \</i> <i>EMIOS_0_CH_16, \</i> <i>EMIOS_0_CH_17, \</i> <i>EMIOS_0_CH_18, \</i> <i>EMIOS_0_CH_19, \</i> <i>EMIOS_0_CH_20, \</i> <i>EMIOS_0_CH_21, \</i> <i>EMIOS_0_CH_22, \</i> <i>EMIOS_0_CH_23, \</i> <i>EMIOS_0_CH_24, \</i> <i>EMIOS_0_CH_25, \</i> <i>EMIOS_0_CH_26, \</i> <i>EMIOS_0_CH_27, \</i> <i>EMIOS_0_CH_28, \</i> <i>EMIOS_0_CH_29</i> <i>EMIOS_0_CH_30</i> <i>EMIOS_0_CH_31, \</i> <i>\}</i>	<i>{ \</i> <i>EMIOS_1_CH_0, \</i> <i>EMIOS_1_CH_1, \</i> <i>EMIOS_1_CH_2, \</i> <i>EMIOS_1_CH_3, \</i> <i>EMIOS_1_CH_4, \</i> <i>EMIOS_1_CH_5, \</i> <i>EMIOS_1_CH_6, \</i> <i>EMIOS_1_CH_7, \</i> <i>EMIOS_1_CH_8, \</i> <i>EMIOS_1_CH_10, \</i> <i>EMIOS_1_CH_16, \</i> <i>EMIOS_1_CH_17, \</i> <i>EMIOS_1_CH_18, \</i> <i>EMIOS_1_CH_19, \</i> <i>EMIOS_1_CH_20, \</i> <i>EMIOS_1_CH_21, \</i> <i>EMIOS_1_CH_22, \</i> <i>EMIOS_1_CH_23, \</i> <i>EMIOS_1_CH_24, \</i> <i>EMIOS_1_CH_25, \</i> <i>EMIOS_1_CH_26, \</i> <i>EMIOS_1_CH_27, \</i> <i>EMIOS_1_CH_28, \</i> <i>EMIOS_1_CH_29, \</i> <i>EMIOS_1_CH_30, \</i> <i>EMIOS_1_CH_31</i>

Table 3-63. Note:The following channels support IPM and IPWM mode .

		\}
--	--	----

3.10.2.19 IcuIndex

Table 3-64. IcuIndex

Description	This parameter Specifies the InstanceId of this module instance. If only one instance is present it shall have the Id 0. NOTE: In current implementation this parameter is not used.
Class	Autosar Parameter
Range	0-255
Default	0
Source File	Icu_Cfg.h
Source Representation	<code>#define ICU_INDEX <0-255></code>
NOTE	<i>This parameter is not used in the current implementation</i>

3.10.2.20 IcuMaxChannel

Table 3-65. IcuMaxChannel

Description	Number of configured icu channels
Class	Autosar Parameter
Range	0-118
Default	0
Source File	Icu_PBCfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ConfigType, ICU_CONST) IcuConfigSet_0 = { 12, /* The number of channels configured*/ Icu_InitPBChannel_0, Icu_InitPBChannelIndex_0, Icu_ChannelId_0, Icu_InitPBHWMap_0, Icu_InitPBHWKMap_0 }; </pre>

3.10.2.21 IcuOverflowNotificationApi

Table 3-66. IcuOveflowNotificationApi

Description	Adds / removes the service Overflow Notification functionality
Class	Autosar Parameter
Range	True,false
Default	False
Source File	Icu_Cfg.h
Source Representation	<code>#define ICU_OVERFLOW_NOTIFICATION_API <STD_OFF, STD_ON></code>

3.10.2.22 IcuReportWakeupSource

Table 3-67. IcuReportWakeupSource

Description	Switch for enabling Wakeup source reporting.
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Icu_Cfg.h
Source Representation	<code>#define ICU_REPORT_WAKEUP_SOURCE <STD_OFF, STD_ON></code>

3.10.2.23 IcuSetModeApi

Table 3-68. IcuSetModeApi

Description	Adds / removes the service Icu_SetMode() from the code
Class	Autosar Parameter
Range	True,false
Default	True
Source File	Icu_Cfg.h
Source Representation	<code>#define ICU_SET_MODE_API <STD_OFF, STD_ON></code>

3.10.2.24 IcuSignalMeasurementApi

Table 3-69. IcuSignalMeasurementApi

Description	Adds / removes the services: Icu_StartSignalMeasurement () and Icu_StopSignalMeasurement ().
Class	Autosar Parameter
Range	True,false

Table continues on the next page...

Table 3-69. IcuSignalMeasurementApi (continued)

Default	True
Source File	Icu_Cfg.h
Source Representation	<code>#define ICU_SIGNAL_MEASUREMENT_API <STD_OFF, STD_ON></code>

3.10.2.25 IcuTimestampApi

Table 3-70. IcuTimestampApi

Description	Adds / removes all services related to the timestamping functionality as listed below from the code: Icu_StartTimestamp(), Icu_StopTimestamp(), Icu_GetTimestampIndex()
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Icu_Cfg.h
Source Representation	<code>#define ICU_TIMESTAMP_API <STD_OFF, STD_ON></code>

3.10.2.26 IcuWakeupFunctionalityApi

Table 3-71. IcuWakeupFunctionalityApi

Description	Adds / removes the service Icu_CheckWakeup() from the code.
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Icu_Cfg.h
Source Representation	<code>#define ICU_WAKEUP_FUNCTIONALITY_API <STD_OFF, STD_ON></code>

3.10.3 Link Time parameters

N/A

3.10.4 Post - Build Parameters

Post-Build parameters, their possible values and their meaning are described in the following text. The Post-Build parameters are implemented as constant structures and arrays stored in flash memory of the ICU.

Configuration Parameters

The configuration screen for ICU postbuild parameters in Tresos® Studio Configuration Tool is given below:

The screenshot shows the 'IcuChannel' configuration window in the Tresos Studio Configuration Tool. The 'Name' field is set to 'IcuChannel_0'. The 'General' tab is selected, displaying a list of configuration parameters for the IcuChannel_0. Each parameter has a corresponding dropdown menu or checkbox to select its value. The parameters include:

- IcuChannelId (0 -> 116): 0
- IcuHwChannel: EMIOS_0_CH_8
- ICU External Enable Interrupt Filter: ☐
- ICU External Interrupt Filter setting (0 -> 15): 0
- ICU Wakeup Pullup Enable: ☐ IcuEmiosFreeze: ☒
- IcuEmiosPrescaler: EMIOS_PRESCALER_DIVIDE_1
- IcuEmiosPrescaler_Alternate: EMIOS_PRESCALER_DIVIDE_1
- IcuEmiosDigitalFilter: EMIOS_DIGITAL_FILTER_BYPASSED
- IcuEmiosBusSelect: EMIOS_BUS_INTERNAL_COUNTER
- IcuDefaultStartEdge: ICU_RISING_EDGE
- IcuMeasurementMode: ICU_MODE_SIGNAL_MEASUREMENT
- IcuDMAEnable: ☐
- IcuUserModeForDutycycle: SAIC
- IcuOverflowNotification: NULL_PTR
- IcuWakeupCapability: ☐ IcuSignalMeasureWithoutInterrupt: ☐
- IcuDisableEcumWakeupSourceNotification: ☐

Figure 3-9. Tresos Plugin snapshot for Icu_Channel form

3.10.4.1 ICU External Interrupt Filter Setting

Table 3-72. Icu_EXT_ISR_IFMCDigitalFilter description

Description	Maximum Interrupt Filter Counter setting
Class	Implementation specific Non-Autosar parameter
Range	[0..15]
Default	0
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ChannelConfigType, ICU_CONST) Icu_InitPBChannel_0[3] ={ /* IcuChannel_0 - IRQ7 */ { ((ICU_RISING_EDGE << ICU_EDGE_PARAM_SHIFT) ICU_EXT_INT_FILTER_ENABLE (6U << ICU_EXT_INT_IFMC_SHIFT), </pre>

Table 3-72. Icu_EXT_ISR_IFMCDigitalFilter description

	<pre> ICU_MODE_SIGNAL_EDGE_DETECT, 0, NULL_PTR, 0U }, [...]</pre>
--	---

3.10.4.2 ICU External Enable Interrupt Filter

Table 3-73. Icu_EXT_ISR_IFERDigitalFilter description

Description	Enable external digital filter counter on the interrupt pads to filter out glitches on the inputs
Class	Implementation specific Non-Autosar parameter
Range	true, false
Default	false
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ChannelConfigType, ICU_CONST) Icu_InitPBChannel_0[3] ={ /* IcuChannel_0 - IRQ7 */ { ((ICU_RISING_EDGE << ICU_EDGE_PARAM_SHIFT) ICU_EXT_INT_FILTER_ENABLE (6U << ICU_EXT_INT_IFMC_SHIFT), ICU_MODE_SIGNAL_EDGE_DETECT, 0, NULL_PTR, 0U }, [...]</pre>

3.10.4.3 IcuChannelWakeupInfo

Table 3-74. IcuChannelWakeupInfo

Description	If the wakeup capability is true the wakeup source referenced is transmitted to the ECU State Manager (Ecum).
--------------------	---

Table continues on the next page...

Table 3-74. IcuChannelWakeupInfo (continued)

Class	Autosar Parameter
Range	NA
Default	NA
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ChannelConfigType, ICU_CONST) Icu_InitPBChannel_0[26]= { { (((Icu_ParamType)ICU_WAKEUP_CAPABLE << ICU_WAKEUP_SHIFT) (ICU_RISING_EDGE << ICU_EDGE_PARAM_SHIFT) (EMIOS_FREEZE_ENABLE << ICU_EMIOS_FREEZE_PARAM_SHIFT) (EMIOS_PRESCALER_DIVIDE_1 << ICU_EMIOS_PRESCALER_PARAM_SHIFT) (EMIOS_DIGITAL_FILTER_BYPASSED << ICU_EMIOS_DIGITAL_FILTER_PARAM_SHIFT) (EMIOS_BUS_A << ICU_EMIOS_BUS_SELECT_PARAM_SHIFT)), ICU_MODE_SIGNAL_EDGE_DETECT, 0U, Icu_Channel_1_Notification, 0U }, } </pre>

3.10.4.4 IcuDefaultStartEdge

Table 3-75. IcuDefaultStartEdge

Description	Configures the default-activation-edge which shall be used for this channel if there was no activation-edge configured by the call of service Icu_SetActivationCondition (). In case the Measurement Mode is "SignalMeasurement" and the properties DutyCycle or Period are set, the edge configured here is used as Default Period Start Edge.
Class	Autosar Parameter
Range	IcuRisingEdge, IcuFallingEdge, IcuBothEdges
Default	IcuRisingEdge
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ChannelConfigType, ICU_CONST) Icu_InitPBChannel_0[26]= { { ((ICU_RISING_EDGE << ICU_EDGE_PARAM_SHIFT) </pre>

Table 3-75. IcuDefaultStartEdge

	<pre> (EMIOS_FREEZE_ENABLE << ICU_EMIOS_FREEZE_PARAM_SHIFT) (EMIOS_PRESCALER_DIVIDE_1 << ICU_EMIOS_PRESCALER_PARAM_SHIFT) (EMIOS_DIGITAL_FILTER_BYPASSED << ICU_EMIOS_DIGITAL_FILTER_PARAM_SHIFT) (EMIOS_BUS_INTERNAL_COUNTER << ICU_EMIOS_BUS_SELECT_PARAM_SHIFT)), ICU_MODE_SIGNAL_MEASUREMENT, ICU_DUTY_CYCLE, NULL_PTR, 0U }, } </pre>
--	--

3.10.4.5 icudisableecumwakeupsourcenotification

Table 3-76. icudisableecumwakeupsourcenotification

Description	Notification to EcuM is disabled true: Notification to EcuM disabled false: Notification to EcuM Enabled
Class	Autosar Parameter
Range	NA
Default	"NULL_PTR"
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ChannelConfigType, ICU_CONST) Icu_InitPBChannel_0[26]= { { (((Icu_ParamType)ICU_WAKEUP_CAPABLE << ICU_WAKEUP_SHIFT) (ICU_RISING_EDGE << ICU_EDGE_PARAM_SHIFT) (EMIOS_FREEZE_ENABLE << ICU_EMIOS_FREEZE_PARAM_SHIFT) (EMIOS_PRESCALER_DIVIDE_1 << ICU_EMIOS_PRESCALER_PARAM_SHIFT) (EMIOS_DIGITAL_FILTER_BYPASSED << ICU_EMIOS_DIGITAL_FILTER_PARAM_SHIFT) (EMIOS_BUS_A << ICU_EMIOS_BUS_SELECT_PARAM_SHIFT)), ICU_MODE_SIGNAL_EDGE_DETECT, 0U, Icu_Channel_1_Notification, FALSE, 0U FALSE, }, </pre>

Table 3-76. icudisableecumwakeupsourcenotification

	...
	}

3.10.4.6 IcuDMAEnable

Table 3-77. IcuDMAEnable

Description	The value for the IcuDmaMaxChannel must match with the number of IcuChannel configured with DMA enabled. For calculating the correct value use the CALC button.
Class	Autosar Parameter
Range	True,false
Default	True
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ConfigType, ICU_CONST) IcuConfigSet_0 = { (((Icu_ParamType)ICU_RISING_EDGE (LS) ICU_EDGE_PARAM_SHIFT) ((Icu_ParamType)EMIOS_UC_SAIC_MODE (LS) ICU_EMIOS_UC_MODE_PARAM_SHIFT) ((Icu_ParamType)EMIOS_PRESCALER_DIVIDE_1 (LS) ICU_EMIOS_PRESCALER_PARAM_SHIFT) ((Icu_ParamType)EMIOS_DIGITAL_FILTER_BYPASSED (LS) ICU_EMIOS_DIG_FILTER_PARAM_SHIFT) ((Icu_ParamType)EMIOS_BUS_INTERNAL_COUNTER (LS) ICU_EMIOS_BUS_SELECT_PARAM_SHIFT)), (Icu_MeasurementModeType)ICU_MODE_SIGNAL_MEASUREMENT, (Icu_MeasurementSubModeType)ICU_DUTY_CYCLE, NULL_PTR, #ifdef (ICU_OVERFLOW_NOTIFICATION_API) (LA) (ICU_OVERFLOW_NOTIFICATION_API == STD_ON) NULL_PTR, #endif /* ICU_OVERFLOW_NOTIFICATION_API */ (boolean)FALSE, 0U, (boolean)FALSE, #ifdef (ICU_DMA_SUPPORTED == STD_ON) (boolean)TRUE, /*Icu with DMA is enabled */ 18U, /*ICU DMA channel*/ #endif }; </pre> Note: LS = Left Shift LA = Logical AND.

3.10.4.7 IcuEmiosBusSelect

Table 3-78. IcuEmiosBusSelect

Description	Selects the counter used with the unified channel.
Class	Autosar Parameter
Range	EMIOS_BUS_A, EMIOS_BUS_DIVERSE, EMIOS_BUS_INTERNAL_COUNTER,
Default	EMIOS_BUS_INTERNAL_COUNTER (if applicable) or EMIOS_BUS_A
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ChannelConfigType, ICU_CONST) Icu_InitPBChannel_0[26]= { { ((ICU_RISING_EDGE << ICU_EDGE_PARAM_SHIFT) (EMIOS_FREEZE_ENABLE << ICU_EMIOS_FREEZE_PARAM_SHIFT) (EMIOS_PRESCALER_DIVIDE_1 << ICU_EMIOS_PRESCALER_PARAM_SHIFT) (EMIOS_DIGITAL_FILTER_BYPASSED << ICU_EMIOS_DIGITAL_FILTER_PARAM_SHIFT) (EMIOS_BUS_INTERNAL_COUNTER << ICU_EMIOS_BUS_SELECT_PARAM_SHIFT)), ICU_MODE_SIGNAL_MEASUREMENT, ICU_DUTY_CYCLE, NULL_PTR, 0U }, ... } </pre>

3.10.4.8 IcuEmiosDigitalFilter

Table 3-79. IcuEmiosDigitalFilter

Description	If a eMIOS channel is being used this option is active, possible values are: 0 (Bypassed), 2, 4, 8, 16 FLT_Clock periods.
Class	Implementation specific Non-Autosar parameter
Range	EMIOS_DIGITAL_FILTER_BYPASSED, EMIOS_DIGITAL_FILTER_02, EMIOS_DIGITAL_FILTER_04, EMIOS_DIGITAL_FILTER_08, EMIOS_DIGITAL_FILTER_16
Default	EMIOS_DIGITAL_FILTER_BYPASSED
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ChannelConfigType, ICU_CONST) Icu_InitPBChannel_0[26]= { </pre>

Table 3-79. IcuEmiosDigitalFilter

	<pre> { ((ICU_RISING_EDGE << ICU_EDGE_PARAM_SHIFT) (EMIOS_FREEZE_ENABLE << ICU_EMIOS_FREEZE_PARAM_SHIFT) (EMIOS_PRESCALER_DIVIDE_1 << ICU_EMIOS_PRESCALER_PARAM_SHIFT) (EMIOS_DIGITAL_FILTER_BYPASSED << ICU_EMIOS_DIGITAL_FILTER_PARAM_SHIFT) (EMIOS_DIGITAL_FILTER_BYPASSED << ICU_EMIOS_DIGITAL_FILTER_PARAM_SHIFT) ICU_MODE_SIGNAL_MEASUREMENT, ICU_DUTY_CYCLE, NULL_PTR, 0U }, ... } </pre>
--	--

3.10.4.9 IcuEmiosFreeze

Table 3-80. IcuEmiosFreeze

Description	If selected eMIOS channel registers are freezed in debug mode.
Class	Implementation specific Non-Autosar parameter
Range	True,false
Default	True
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ChannelConfigType, ICU_CONST) Icu_InitPBChannel_0[26]= { (ICU_FREEZE_ENABLE EMIOS_PRESCALER_DIVIDE_1 ICU_DIGITAL_FILTER_BYPASSED ICU_FILTER_PRESCALE EMIOS_BUS_INTERNAL_COUNTER ICU_RISING_EDGE_PARAM ICU_PRES_EN), ICU_MODE_SIGNAL_EDGE_DETECT, 0, EMIOS_0_CH_0_Notification, 0 } </pre>

3.10.4.10 IcuEmiosPrescaler_Alternate

Table 3-81. IcuEmiosPrescaler_Alternate

Description	If an eMIOS channel is being used, this configures the alternate prescaler value for the specific channel.
Class	Implementation specific Non-Autosar parameter
Range	EMIOS_PRESCALER_DIVIDE_1, EMIOS_PRESCALER_DIVIDE_2, EMIOS_PRESCALER_DIVIDE_3, EMIOS_PRESCALER_DIVIDE_4
Default	EMIOS_PRESCALER_DIVIDE_1
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ChannelConfigType, ICU_CONST) Icu_InitPBChannel_0[1] = { { ((ICU_RISING_EDGE (1s) ICU_EDGE_PARAM_SHIFT) (EMIOS_FREEZE_ENABLE (1s) ICU_EMIOS_FREEZE_PARAM_SHIFT) (EMIOS_PRESCALER_DIVIDE_1 (1s) ICU_EMIOS_PRESCALER_PARAM_SHIFT) (EMIOS_PRESCALER_DIVIDE_1 (1s) ICU_EMIOS_PRESC_ALT_PARAM_SHIFT) (EMIOS_DIGITAL_FILTER_BYPASSED (1s) ICU_EMIOS_DIGITAL_FILTER_PARAM_SHIFT) (EMIOS_BUS_INTERNAL_COUNTER (1s) ICU_EMIOS_BUS_SELECT_PARAM_SHIFT)), ICU_MODE_SIGNAL_MEASUREMENT, ICU_DUTY_CYCLE, NULL_PTR, 0U } } </pre>

3.10.4.11 IcuEmiosPrescaler

Table 3-82. IcuEmiosPrescaler

Description	If an eMIOS channel is being used, this configures the prescaler value for the specific channel.
Class	Implementation specific Non-Autosar parameter
Range	EMIOS_PRESCALER_DIVIDE_1, EMIOS_PRESCALER_DIVIDE_2, EMIOS_PRESCALER_DIVIDE_3, EMIOS_PRESCALER_DIVIDE_4
Default	EMIOS_PRESCALER_DIVIDE_1
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ChannelConfigType, ICU_CONST) Icu_InitPBChannel_0[26]= { { ((ICU_RISING_EDGE << ICU_EDGE_PARAM_SHIFT) (EMIOS_FREEZE_ENABLE << ICU_EMIOS_FREEZE_PARAM_SHIFT) </pre>

Table 3-82. IcuEmiosPrescaler

	<pre> (EMIOS_PRESCALER_DIVIDE_1 << ICU_EMIOS_PRESCALER_PARAM_SHIFT) (EMIOS_DIGITAL_FILTER_BYPASSED << ICU_EMIOS_DIGITAL_FILTER_PARAM_SHIFT) (EMIOS_BUS_INTERNAL_COUNTER << ICU_EMIOS_BUS_SELECT_PARAM_SHIFT)), ICU_MODE_SIGNAL_MEASUREMENT, ICU_DUTY_CYCLE, NULL_PTR, 0U }, ... } </pre>
--	--

3.10.4.12 IcuMeasurementMode

Table 3-83. IcuMeasurementMode

Description	Configures the measurement mode of this channel.
Class	Autosar Parameter
Range	IcuSignalEdgeDetection, IcuEdgeCounter, IcuSignalMeasurement, IcuTimestamp
Default	IcuSignalEdgeDetection
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ChannelConfigType, ICU_CONST) Icu_InitPBChannel_0[26]= { { ((ICU_RISING_EDGE << ICU_EDGE_PARAM_SHIFT) (EMIOS_FREEZE_ENABLE << ICU_EMIOS_FREEZE_PARAM_SHIFT) (EMIOS_PRESCALER_DIVIDE_1 << ICU_EMIOS_PRESCALER_PARAM_SHIFT) (EMIOS_DIGITAL_FILTER_BYPASSED << ICU_EMIOS_DIGITAL_FILTER_PARAM_SHIFT) (EMIOS_BUS_INTERNAL_COUNTER << ICU_EMIOS_BUS_SELECT_PARAM_SHIFT)), ICU_MODE_SIGNAL_MEASUREMENT, ICU_DUTY_CYCLE, NULL_PTR, 0U }, } </pre>

3.10.4.13 Icu Overflow Notification

Table 3-84. IcuOverflowNotification

Description	Icu Overflow Notification Handle. In order to activate this field you have to: • enable <i>IcuOverflowNotificationApi</i>, • choose one of the modes: • ICU_MODE_EDGE_COUNTER, • ICU_MODE_SIGNAL_MEASUREMENT, • ICU_MODE_TIMESTAMP
Class	Autosar Parameter
Range	NA
Default	"NULL_PTR"
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ChannelConfigType, ICU_CONST) Icu_InitPBChannel_0[3] ={ /* IcuChannel_0 - EMIOS_0_CH_3 */ { ((ICU_RISING_EDGE << ICU_EDGE_PARAM_SHIFT) (ICU_EMIOS_PRESCALER_DIVIDE_128 << ICU_EMIOS_PRESCALER_SHIFT) (0U << ICU_EMIOS_FILTER_CNT_SHIFT) (0U << ICU_EMIOS_FILTER_PER_SHIFT)), ICU_MODE_SIGNAL_MEASUREMENT, ICU_DUTY_CYCLE, NULL_PTR, #if defined(ICU_OVERFLOW_NOTIFICATION_API) AND (ICU_OVERFLOW_NOTIFICATION_API == STD_ON) NULL_PTR, #endif/* ICU_OVERFLOW_NOTIFICATION_API */ 0U }, [...]</pre>

3.10.4.14 Icu_ParamType

Icu_ParamType is defined as a uint32. The Icu_ParamValue contains combined bit fields for initialization options, for different registers. Definitions are provided in Icu_cfg.h.

The table below depicts the Icu_ParamType values.

Table 3-85. Icu_ParamType Values

Definition	Description
ICU_FALLING_EDGE	Initialization option for eMIOS, IRQ and WKUP channels. It is the Default start edge when falling edge is selected.
ICU_RISING_EDGE	Initialization option for eMIOS, IRQ and WKUP channels. It is the Default start edge when rising edge is selected.
ICU_BOTH_EDGES	Initialization option for eMIOS, IRQ and WKUP channels. It is the Default start edge when both edges are selected.
ICU_WAKEUP_CAPABLE	Initialization option to specify the channel as wakeup capable. This works for eMIOS, IRQ and WKUP channels
EMIOS_BUS_A	Initialization option to specify that the channel runs off Timer Bus A. Hardware Dependent Definition. DO NOT CHANGE.
EMIOS_BUS_DIVERSE	Initialization option to specify that the channels 0 to 7 run off Timer Bus B, Channels 8 to 15 run off from Timer bus C. Hardware Dependent Definition. DO NOT CHANGE.
EMIOS_BUS_INTERNAL_COUNTER	Initialization option to specify that the channel runs off its own internal time base. Hardware Dependent Definition. DO NOT CHANGE.
EMIOS_DIGITAL_FILTER_BYPASSED	Initialization option to disable the filter. Hardware Dependent Definition. DO NOT CHANGE.
EMIOS_DIGITAL_FILTER_02	Initialization option to set the filter to 2 clocks. Hardware Dependent Definition. DO NOT CHANGE.
EMIOS_DIGITAL_FILTER_04	Initialization option to set the filter to 4 clocks. Hardware Dependent Definition. DO NOT CHANGE.
EMIOS_DIGITAL_FILTER_08	Initialization option to set the filter to 8 clocks. Hardware Dependent Definition. DO NOT CHANGE.
EMIOS_DIGITAL_FILTER_16	Initialization option to set the filter to 16 clocks. Hardware Dependent Definition. DO NOT CHANGE.
EMIOS_PRESCALER_DIVIDE_1	Initialization option to set the channel prescale value to 1. Only applies to channels configured to run off the internal counter bus. Hardware Dependent Definition. DO NOT CHANGE.
EMIOS_PRESCALER_DIVIDE_2	Initialization option to set the channel prescale value to 2. Only applies to channels configured to run off the internal counter bus. Hardware Dependent Definition. DO NOT CHANGE.
EMIOS_PRESCALER_DIVIDE_3	Initialization option to set the channel prescale value to 3. Only applies to channels configured to run off the internal counter bus. Hardware Dependent Definition. DO NOT CHANGE.
EMIOS_PRESCALER_DIVIDE_4	Initialization option to set the channel prescale value to 4. Only applies to channels configured to run off the internal counter bus. Hardware Dependent Definition. DO NOT CHANGE.
EMIOS_FREEZE_ENABLE	Initialization option to enable freeze mode for the channel. If this option is not specified, freeze mode will be disabled when Icu_Init is called for the channel. Hardware Dependent Definition. DO NOT CHANGE.
SIUL_INT_FILTER_ENABLE	Initialization option to enable IRQ filter
WKPU_INT_FILTER_ENABLE	Initialization option for analog filter on the corresponding interrupt pads to filter out glitches on the inputs
IcuEXT_ISR_IFMCDigitalFilter	Initialization option to configure the filter counter associated with each digital glitch filter.
WKPU_PULLUP_ENABLE	Initialization option to enable a pullup on the corresponding interrupt pads to pull an unconnected wakeup/interrupt input to a value of '1'

3.10.4.15 IcuSignalMeasurementProperty

Table 3-86. IcuSignalMeasurementProperty

Description	Configures the property that could be measured in case the mode is "IcuSignalMeasurement". This property can not be changed during runtime.
Class	Autosar Parameter
Range	IcuPeriodTime, IcuDutyCycle, IcuHighTime, IcuLowTime
Default	IcuDutyCycle
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ChannelConfigType, ICU_CONST) Icu_InitPBChannel_0[26]= { { ((ICU_RISING_EDGE << ICU_EDGE_PARAM_SHIFT) (EMIOS_FREEZE_ENABLE << ICU_EMIOS_FREEZE_PARAM_SHIFT) (EMIOS_PRESCALER_DIVIDE_1 << ICU_EMIOS_PRESCALER_PARAM_SHIFT) (EMIOS_DIGITAL_FILTER_BYPASSED << ICU_EMIOS_DIGITAL_FILTER_PARAM_SHIFT) (EMIOS_BUS_INTERNAL_COUNTER << ICU_EMIOS_BUS_SELECT_PARAM_SHIFT)), ICU_MODE_SIGNAL_MEASUREMENT, ICU_LOW_TIME, NULL_PTR, 0U }, }, </pre>

3.10.4.16 IcuSignalMeasureWithoutInterrupt

Table 3-87. IcuSignalMeasureWithoutInterrupt

Description	Measurement of ICU signal property without using interrupt, this functionality is implemented for IPM and IPWM mode channels only. Also this feature is not supported for DUTY CYCLE measurement using IPM and IPWM channels
Class	Non Autosar Parameter
Range	True, False
Default	False
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ChannelConfigType, ICU_CONST) </pre>

Table 3-87. IcuSignalMeasureWithoutInterrupt

	<pre> Icu_InitPBChannel_0[26]= { { ((ICU_RISING_EDGE << ICU_EDGE_PARAM_SHIFT) (EMIOS_FREEZE_ENABLE << ICU_EMIOS_FREEZE_PARAM_SHIFT) (EMIOS_PRESCALER_DIVIDE_1 << ICU_EMIOS_PRESCALER_PARAM_SHIFT) (EMIOS_DIGITAL_FILTER_BYPASSED << ICU_EMIOS_DIGITAL_FILTER_PARAM_SHIFT) (EMIOS_BUS_INTERNAL_COUNTER << ICU_EMIOS_BUS_SELECT_PARAM_SHIFT)), ICU_MODE_SIGNAL_MEASUREMENT, ICU_DUTY_CYCLE, NULL_PTR, False 0U }, ... } </pre>
--	---

3.10.4.17 IcuSignalNotification

Table 3-88. IcuSignalNotification

Description	Notification function for signal notification.
Class	Autosar Parameter
Range	NA
Default	"NULL"
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ChannelConfigType, ICU_CONST Icu_InitPBChannel_0[26]= { { (((Icu_ParamType)ICU_WAKEUP_CAPABLE << ICU_WAKEUP_SHIFT) (ICU_RISING_EDGE << ICU_EDGE_PARAM_SHIFT) (EMIOS_FREEZE_ENABLE << ICU_EMIOS_FREEZE_PARAM_SHIFT) (EMIOS_PRESCALER_DIVIDE_1 << ICU_EMIOS_PRESCALER_PARAM_SHIFT) (EMIOS_DIGITAL_FILTER_BYPASSED << ICU_EMIOS_DIGITAL_FILTER_PARAM_SHIFT) (EMIOS_BUS_A << ICU_EMIOS_BUS_SELECT_PARAM_SHIFT)), ICU_MODE_SIGNAL_EDGE_DETECT, </pre>

Table 3-88. IcuSignalNotification

	<pre> 0U, Icu_Channel_1_Notification, 0U }, } </pre>
--	--

3.10.4.18 IcuTimestampMeasurementProperty

Table 3-89. IcuTimestampMeasurementProperty

Description	Configures the handling of the buffer in case the mode is "Timestamp"
Class	Autosar Parameter
Range	IcuLinearBuffer, IcuCircularBuffer
Default	IcuLinearBuffer
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ChannelConfigType, ICU_CONST) Icu_InitPBChannel_0[26]= { { ((ICU_RISING_EDGE << ICU_EDGE_PARAM_SHIFT) (EMIOS_FREEZE_ENABLE << ICU_EMIOS_FREEZE_PARAM_SHIFT) (EMIOS_PRESCALER_DIVIDE_1 << ICU_EMIOS_PRESCALER_PARAM_SHIFT) (EMIOS_DIGITAL_FILTER_BYPASSED << ICU_EMIOS_DIGITAL_FILTER_PARAM_SHIFT) (EMIOS_BUS_A << ICU_EMIOS_BUS_SELECT_PARAM_SHIFT)), ICU_MODE_TIMESTAMP, ICU_LINEAR_BUFFER, Icu_TimestampNotification, 0U }, ... }, </pre>

3.10.4.19 IcuTimestampNotification

Table 3-90. IcuTimestampNotification

Description	Notification function if the number of requested timestamps (Notification interval > 0) are acquired.
Class	Autosar Parameter
Range	NA
Default	NULL_PTR
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ChannelConfigType, ICU_CONST) Icu_InitPBChannel_0[26]= { { ((ICU_RISING_EDGE << ICU_EDGE_PARAM_SHIFT) (EMIOS_FREEZE_ENABLE << ICU_EMIOS_FREEZE_PARAM_SHIFT) (EMIOS_PRESCALER_DIVIDE_1 << ICU_EMIOS_PRESCALER_PARAM_SHIFT) (EMIOS_DIGITAL_FILTER_BYPASSED << ICU_EMIOS_DIGITAL_FILTER_PARAM_SHIFT) (EMIOS_BUS_A << ICU_EMIOS_BUS_SELECT_PARAM_SHIFT)), ICU_MODE_TIMESTAMP, ICU_LINEAR_BUFFER, Icu_TimestampNotification, 0U }, } </pre>

3.10.4.20 IcuUserModeForDutycycle

Table 3-91. IcuUserModeForDutycycle

Description	This parameter will be enabled in configuration only when IcuSignalMeasurementProperty is ICU_DUTY_CYCLE.
Class	Non Autosar Parameter
Range	True,false
Default	False
Source File	Icu_PBcfg.c, Icu_Cfg.c
Source Representation	<pre> #define ICU_MODE_SIGNAL_MEASUREMENT <STD_OFF, STD_ON>, #define ICU_DUTY_CYCLE <STD_OFF, STD_ON> </pre>

3.10.4.21 IcuWakeupCapability

Table 3-92. IcuWakeupCapability

Description	Boolean flag whether wake-up on this channel is supported or not.
Class	Autosar Parameter
Range	True,false
Default	True
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ChannelConfigType, ICU_CONST) Icu_InitPBChannel_0[26]= { { (((Icu_ParamType)ICU_WAKEUP_CAPABLE << ICU_WAKEUP_SHIFT) (ICU_RISING_EDGE << ICU_EDGE_PARAM_SHIFT) (EMIOS_FREEZE_ENABLE << ICU_EMIOS_FREEZE_PARAM_SHIFT) (EMIOS_PRESCALER_DIVIDE_1 << ICU_EMIOS_PRESCALER_PARAM_SHIFT) (EMIOS_DIGITAL_FILTER_BYPASSED << ICU_EMIOS_DIGITAL_FILTER_PARAM_SHIFT) (EMIOS_BUS_A << ICU_EMIOS_BUS_SELECT_PARAM_SHIFT)), ICU_MODE_SIGNAL_EDGE_DETECT, 0U, Icu_Channel_1_Notification, 0U }, ... } </pre>

3.10.4.22 IcuWKPU_ISR_WIPUER

Table 3-93. IcuWKPU_ISR_WIPUER

Description	Wakeup/Interrupt Pullup Enable Register is used to enable a pullup on the corresponding interrupt pads to pull an unconnected wakeup/interrupt input to a value of 1
Class	Non-autosar Parameter
Range	True,false
Default	false
Source File	Icu_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> CONST(Icu_ChannelConfigType, ICU_CONST) Icu_InitPBChannel_0[26]= { </pre>

Table 3-93. IcuWKPU_ISR_WIPUE
R

	<pre>{ (((Icu_ParamType)ICU_WAKEUP_CAPABLE << ICU_WAKEUP_SHIFT) (ICU_RISING_EDGE << ICU_EDGE_PARAM_SHIFT) (EMIOS_FREEZE_ENABLE << ICU_EMIOS_FREEZE_PARAM_SHIFT) (EMIOS_PRESCALER_DIVIDE_1 << ICU_EMIOS_PRESCALER_PARAM_SHIFT) (EMIOS_DIGITAL_FILTER_BYPASSED << ICU_EMIOS_DIGITAL_FILTER_PARAM_SHIFT) (WKPU_PULLUP_ENABLE << ICU_WKPU_PULLUP_ENABLE_PARAM_SHIFT) (EMIOS_BUS_A << ICU_EMIOS_BUS_SELECT_PARAM_SHIFT)), ICU_MODE_SIGNAL_EDGE_DETECT, 0U, Icu_Channel_1_Notification, 0U }, ... }</pre>
--	--

How to Reach Us:**Home Page:**freescale.com**Web Support:**freescale.com/support

Information in this document is provided solely to enable system and software implementers to use Freescale products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document.

Freescale reserves the right to make changes without further notice to any products herein. Freescale makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does Freescale assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in Freescale data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. Freescale does not convey any license under its patent rights nor the rights of others. Freescale sells products pursuant to standard terms and conditions of sale, which can be found at the following address: freescale.com/SalesTermsandConditions.

Freescale, the Freescale logo, Altivec, C-5, CodeTest, CodeWarrior, ColdFire, ColdFire+, C-Ware, Energy Efficient Solutions logo, Kinetis, mobileGT, PowerQUICC, Processor Expert, QorIQ, Qorivva, StarCore, Symphony, and VortiQa are trademarks of Freescale Semiconductor, Inc., Reg. U.S. Pat. & Tm. Off. Airfast, BeeKit, BeeStack, CoreNet, Flexis, Layerscape, MagniV, MXC, Platform in a Package, QorIQ Qonverge, QUICC Engine, Ready Play, SafeAssure, SafeAssure logo, SMARTMOS, Tower, TurboLink, Vybrid, and Xtrinsic are trademarks of Freescale Semiconductor, Inc. All other product or service names are the property of their respective owners.

© 2013 Freescale Semiconductor, Inc.

Document Number UM131CUASR4.0R1.0.1
Revision 1.2